

DISENTANGLING THE CAUSES OF PLASTICITY LOSS IN NEURAL NETWORKS

Clare Lyle[†] Zeyu Zheng[†] Khimya Khetarpal[†] James Martens[†] Hado van Hasselt[†] Razvan Pascanu[†]

Will Dabney[†]

ABSTRACT

Underpinning the past decades of work on the design, initialization, and optimization of neural networks is a seemingly innocuous assumption: that the network is trained on a *stationary* data distribution. In settings where this assumption is violated, e.g. deep reinforcement learning, learning algorithms become unstable and brittle with respect to hyperparameters and even random seeds. One factor driving this instability is the loss of plasticity, meaning that updating the network’s predictions in response to new information becomes more difficult as training progresses. While many recent works provide analyses and partial solutions to this phenomenon, a fundamental question remains unanswered: to what extent do known mechanisms of plasticity loss overlap, and how can mitigation strategies be combined to best maintain the trainability of a network? This paper addresses these questions, showing that loss of plasticity can be decomposed into multiple independent mechanisms and that, while intervening on any single mechanism is insufficient to maintain trainability in all cases, intervening on multiple mechanisms in conjunction results in highly robust learning algorithms. We show that a combination of layer normalization and weight decay is highly effective at maintaining plasticity in a variety of synthetic nonstationary learning tasks, and further demonstrate its effectiveness on naturally arising nonstationarities, including reinforcement learning in the Arcade Learning Environment.

1 INTRODUCTION

Training a neural network on a single task is relatively straightforward: for popular data modalities it is usually possible to take an off-the-shelf architecture and optimization algorithm and, with minimal hyperparameter tuning, obtain a reasonably performant model. Yet the statistical relationships modeled by machine learning systems are often dynamic: consumer preferences change over time, information on the internet goes out of date, and words enter and leave common usage. In reinforcement learning systems, the very act of improving the learner’s policy introduces changes in the distribution of data it collects for further training. Without the ability to update its predictions in response to these changes, a learning system’s performance will inevitably plateau or even degrade. Such declines in performance are often resolved in practice by resetting the model parameters and training from scratch on updated training data, but this solution introduces additional computational overhead which can be prohibitively expensive for large models. It is therefore desirable to avoid the need for parameter resets by maintaining the network’s ability to adapt to new learning signals throughout the course of training.

The observation that training a neural network on some learning problems results in a reduced ability to adapt to new tasks has been made independently several times, both in reinforcement learning (Dohare et al., 2021; Igl et al., 2021; Kumar et al., 2020; Lyle et al., 2021) and in supervised learning (Ash & Adams, 2020; Berariu et al., 2021). While this phenomenon, which we will refer to in this paper as *plasticity loss*, is quite prevalent, our understanding of why it occurs remains vague. Lyle et al. (2023) show a number of negative results indicating that we cannot attribute all instances of plasticity loss to a single measured quantity such as parameter norm or number of dead units. As a result, methods which aim to preserve plasticity typically either target a single pathology (Sokar et al., 2023; Abbas et al., 2023), which runs the risk of allowing plasticity loss to occur via other unaddressed mechanisms, or regularize more abstract properties of the network towards their initial value (Lyle et al., 2021; Lewandowski et al., 2023; Kumar et al., 2023b), which can run the risk of interfering with the primary objective. A useful model of plasticity loss should combine the best of both of these approaches, allowing for the development of principled mitigation strategies which target several independent mechanisms in conjunction.

[†] Google DeepMind. Correspondence to clarelyle@google.com

This paper aims to develop such a model, and use it to guide the development of more effective methods to maintain plasticity. We arrive at the conclusion that, although any single statistic of the network is unable to explain loss of plasticity in full generality, a handful of easily-identified pathologies explain many known instances of plasticity loss. While we do not aim to provide an exhaustive list of all network pathologies that can lead to loss of trainability, we show that plasticity loss can in fact be driven by multiple *independent* mechanisms, in the sense that an intervention which acts on one such mechanism may still allow plasticity loss to occur by another.

1. **Regression target magnitude:** we find that regressing on targets with a large mean relative to their variance can result in training instabilities when networks are fine-tuned on new regression targets. This observation suggests that much of the instability observed in deep RL agents is due to the *scale* of the TD targets, rather than their inherent non-stationarity.
2. **Preactivation distribution shift:** networks trained under non-stationary objectives are vulnerable to distribution shifts in their pre-activations, leading to reduced expressivity as the activation acts as a linear (or even zero, in the case of dead ReLUs) function.
3. **Unbounded growth of the parameter norm:** extreme parameter norm values can lead to a variety of pathologies, resulting in large gradient norms in unnormalized networks, or reduced effective update magnitudes in the case of normalized networks.

We proceed to take these insights to develop a ‘Swiss cheese model’ of mitigation strategies, and show that interventions which improve robustness to preactivation distribution shift, regression target magnitude, and parameter growth can be studied independently and then combined to give additive benefit. Whereas prior work indicated that each mechanism on its own cannot fully explain plasticity loss (Lyle et al., 2023), we find that addressing several mechanisms in conjunction is sufficient to yield negligible loss of plasticity in a variety of synthetic benchmarks, and that these methods can also improve performance on both deep reinforcement learning benchmarks and on natural distribution shift datasets. By identifying the best interventions for each mechanism in isolation and then combining them, we can significantly reduce the combinatorial complexity of the search over mitigation strategies, a finding which has exciting implications for future work on the stabilization of optimization in nonstationary learning problems.

2 BACKGROUND AND RELATED WORK

Our analysis in this paper is grounded in two related branches of the neural network literature. The first is the study of what properties make a network trainable at initialization. The second is a study of how networks can deviate from these trainable initializations over the course of training, and as a result lose plasticity.

2.1 NETWORK TRAINING, SIGNAL PROPAGATION, AND PREACTIVATION DISTRIBUTIONS

The training dynamics of neural networks have been extensively studied over the past decades (e.g. Sutskever et al., 2013). This literature has highlighted a number of important design choices for neural network architectures, initialization schemes, and optimizers, which will come to bear on the analysis in this paper. Of particular relevance to this work is layer normalization (Ba et al., 2016), which normalizes vectors in a network (often preactivations) to have sample mean 0 and variance 1, commonly used activation functions like ReLU and tanh, and schemes designed to achieve good signal propagation¹ via careful initialization (Poole et al., 2016; Balduzzi et al., 2017) or network design and transformation (Martens et al., 2021; Hayou et al., 2021; Zhang et al., 2021; He et al., 2023).

In the signal propagation literature (e.g. Daniely et al., 2016; Poole et al., 2016; Schoenholz et al., 2017; Lee et al., 2017; Xiao et al., 2018; Hanin & Nica, 2019; Hayou et al., 2019; Yang, 2019; Martens et al., 2021), the distribution of preactivations (i.e. the inputs to element-wise nonlinear layers) plays a crucial role in predicting the functional behavior, and ultimate trainability, of a randomly initialized network. In the most popular schemes to achieve good signal propagation, parameters are initialized as i.i.d. Gaussian with mean and variance chosen so that the empirical preactivation distribution within a layer resembles a Gaussian with some target mean μ and variance σ^2 (with high probability for all network inputs). The choices of μ and σ^2 are crucial, and their optimal values will depend on the characteristics of the activation function, as well as those of the network’s architecture (Martens et al., 2021). While there can be no universally optimal choice² for μ and σ^2 , the standard choice $\mu = 0$ and $\sigma^2 = 1$ will often be good

¹Roughly speaking, networks with poor signal propagation do not do a good job of propagating information about their inputs to deeper layers. This can lead to optimization difficulties, such as vanishing or exploding gradients, as has been formalized in Xiao et al. (2020) and Martens et al. (2021) in the “NTK regime”.

²To see this, note that if μ_0 and σ_0^2 are optimal for a given activation ϕ_0 , then $\mu_1 = \mu_0 + 3$ and $\sigma_1^2 \equiv \sigma_0^2/10$ would be optimal for the activation defined by $\phi_1(x) \equiv \phi_0(\sqrt{10}(x - 3))$.

enough for many commonly used activation functions, assuming modest depth or the use of residual connections (He et al., 2016). Martens et al. (2021) and Zhang et al. (2021) even give methods for transforming any activation function to work well with $\mu = 0$ and $\sigma^2 = 1$. Notably, applying layer normalization to a preactivation vector ensures that the 1st and 2nd order sample statistics precisely agree with $\mu = 0$ and $\sigma^2 = 1$. While this would not guarantee the maintenance of good signal propagation as training progresses, as this depends on various other conditions holding, it may indeed be helpful. Indeed, prior works (Lyle et al., 2023; Kumar et al., 2023a) have observed that networks which incorporate normalization layers tend to be better able to maintain plasticity.

2.2 LOSS OF PLASTICITY

A good initialization ensures that, at least when training begins, the network it is applied to is capable of learning – in other words, that it is *plastic*. As training progresses, however, it is possible for this plasticity to be lost (Dohare et al., 2021; Lyle et al., 2021; Berariu et al., 2021; Abbas et al., 2023). The term plasticity has been used to refer to both a network’s ability to attain low generalization error (Berariu et al., 2021), and its ability to improve its performance on a training set (Abbas et al., 2023; Nikishin et al., 2023; Kumar et al., 2023b). This paper will adopt the latter interpretation, focusing on mitigating the optimization difficulties observed in nonstationary learning problems. Specifically, in this work we will say that a network has lost plasticity if it is unable to optimize its objective function as effectively as a randomly initialized network. Plasticity can thus be thought of as the quality of a particular point in parameter space to serve as a starting point for optimization. It can be useful to also include the optimizer state in this definition, as illustrated by Asadi et al. (2023). In this paper, we use the terms plasticity and trainability interchangeably, in line with the usage of Lyle et al. (2023) as opposed to Berariu et al. (2021). Notably, this definition does not imply that a loss of plasticity is permanent – it is certainly in principle possible for the network to enter an ill-conditioned region of the loss landscape where optimization is slow and then escape. Even a transient reduction in trainability, however, can have serious implications on the performance of learning algorithms in contexts such as deep RL, and so it is desirable to avoid these regions.

Several works have studied the mechanisms of plasticity loss and proposed novel mitigation strategies (Ash & Adams, 2020; Dohare et al., 2021; Nikishin et al., 2022; Abbas et al., 2023; Lee et al., 2023). In reinforcement learning, additional analysis has gone into identifying instabilities due to bootstrapping which may exacerbate this phenomenon (Kumar et al., 2020). While some prior works have suggested that TD targets may induce overfitting in neural networks (Raileanu & Fergus, 2021; Lyle et al., 2022), growth of their scale over the course of training has not been considered as a possible impediment to optimization outside of multi-task settings (van Hasselt et al., 2016). Despite several analyses proposing potential explanations of plasticity loss, the work of Lyle et al. (2023) suggests that none of these mechanisms constitutes the sole determinant of network trainability. The key insight provided by this work is that, to a large extent, several key mechanisms of plasticity loss are independent (we provide a more detailed discussion of this notion of independence in Appendix G) and so mitigating one of these mechanisms does not entail mitigating the others.

3 A DEEPER LOOK INTO PLASTICITY

In this section we aim to decompose the process of plasticity loss into three components: the learning dynamics induced by a task, the effect of these dynamics on the parameters and features of the network, and the manifestation of these changes in networks which have lost plasticity. We will conclude from this analysis that while plasticity loss can occur via a variety of independent mechanisms, it tends to produce similar pathologies in the empirical neural tangent kernel (which, as we will argue, are predictive of training difficulties).

3.1 WHAT KINDS OF LEARNING PROBLEMS INDUCE PLASTICITY LOSS?

There are many situations in which loss of plasticity does not prevent a network from minimizing its objective function – if this were not the case, we would not have seen the widespread adoption of deep learning in fields such as computer vision and natural language processing. This then raises the question: what properties of a learning problem *do* cause a neural network to lose plasticity? In this subsection, we outline two independent factors which can be demonstrated to induce plasticity loss in neural networks: the magnitude of the regression target mean in regression problems, and the smoothness of the distribution shift induced by the nonstationarity. Though we do not claim that these two factors exclusively characterize all problems which might impair trainability, their independence is an important theme that will recur throughout this paper.

Regression target scale. The first property we will consider is motivated by the analysis of Lyle et al. (2023), who observed increased optimization difficulty as a function of training steps in a DQN agent trained on a trivially sim-

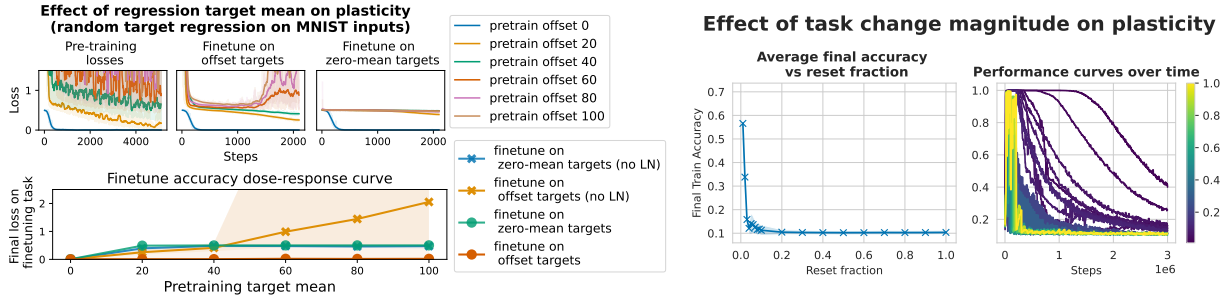


Figure 1: **Left.** Illustration of the relationship between pretraining target magnitude and optimization speed on a new task. We see a strong dose-response effect from increasing the magnitude of regression targets on the final loss on a fine-tuning task, and observe similar trends in the learning curves on these tasks as those observed by [Lyle et al. \(2023\)](#) in DQN agents trained on contextual bandits. **Right.** Dose-response curves for the effect of target offset scale and the magnitude of the distribution shift on plasticity. We see that more severe distribution shifts, e.g. randomizing an entire dataset simultaneously rather than gradually, results in more extreme loss of plasticity.

ple contextual bandit environment, whose observation space was an image classification dataset, and whose reward function gave reward 1 whenever the selected action corresponded to the label of the observation. The resulting non-stationarity induced by training a DQN agent on such a task is highly structured and, depending on the target prediction initialization, can exhibit dynamics similar to regressing on one-hot image labels and periodically adding a constant offset to the regression targets. We claim that it is this deceptively simple structure to the nonstationarity which drove the optimization difficulties encountered by this agent, i.e. that plasticity loss is due to the *monotone increase* in the target mean, rather than its nonstationarity. To support this claim, we show that similar loss of plasticity can be induced by simply regressing neural networks on targets with large means in a *stationary* learning problem.

To do so, we construct a fixed regression problem consisting of random output targets offset by some fixed constant, using as input images from the MNIST dataset. We use this learning problem as a pretraining task, and then fine-tune the network on a new set of random target outputs, offset by either the same constant or zero. We observe in Figure 1 (leftmost column and top right subplot) that pretraining on large offsets significantly reduces the network’s ability to learn on new targets, whether these new targets have the same mean as the pretraining task or are centered at zero, and that the degree of interference with new tasks increases as a function of the pretraining offset, inducing a dose-response effect. While layer normalization can mitigate this trend, it does not completely eliminate it, particularly in the case of centered fine-tuning targets. In Appendix F.1 we provide additional analysis which suggests that the primary mechanism for this loss of plasticity is that the network learns to encode the offset in a single-dimensional subspace of the feature space, leading to poorly-conditioned feature matrices as the bias encoding dwarfs the other components of the features. As we will see in Section 3.3, an analogous ill-conditioning effect occurs in the empirical NTK of networks trained on these tasks as well.

Smoothness of distribution shifts. A simple nonstationary task in which one can quickly induce loss of plasticity is the sequential memorization of randomized labels. It has been argued in prior work that the sudden distribution shift in gradient magnitudes induced by these rapid task changes is responsible for much of the catastrophic loss of plasticity which has been observed in these domains ([Lyle et al., 2023](#)). We construct a simple experiment to explore how sensitive the plasticity loss phenomenon is to the suddenness of this distribution shift by training a CNN (architecture details in Appendix C) on a sequence of randomly generated labels of the CIFAR-10 image classification dataset. At each iteration, we re-randomize some fraction $\epsilon \in [0.01, 1]$ of these labels, and continue training from the network’s current parameters. We train each network for the equivalent of 40 iterations of 100,000 steps (applying resets more frequently for smaller re-randomization fractions to attain the same total number of individual labels which are reset over the experiment). We use three activation functions, ReLU, GeLU, and Leaky ReLU (with negative slope of 0.01).

We see in the top leftmost subplot of Figure 1 that more sudden task changes result in more severe loss of plasticity, as evidenced by the downward trajectory as the fraction of the dataset which is reset grows in the top leftmost inset. In the right hand side, we visualize the trajectories of these networks over several iterations, observing that larger re-randomization fractions lead to more precipitous declines in networks, even keeping the total number of label resets over the whole dataset constant. While larger, infrequent relabellings are worse for plasticity, even the regime of resetting only 1% of labels at a time induces a gradual decline in accuracy. In the next section, we will explore the implications of these observations and take a fine-grained look into precisely how label re-randomization causes networks to become less trainable.

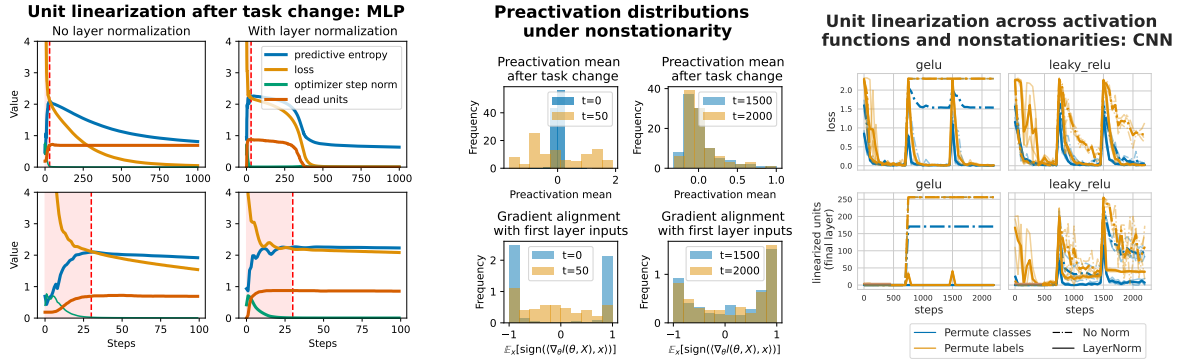


Figure 2: **Accumulation of dead units after a task change:** we visualize the learning dynamics of a small MLP trained to memorize random labels of the MNIST dataset immediately after a task change. **Left.** We highlight an early phase of training wherein predictive entropy sharply increases along with the number of dead units. **Middle.** The distribution shift in the pre-activations in this early phase, and the concomitant spike in gradients with negative dot product with *all* incoming features at the start of learning, contrasting with the stable dynamics later in training. **Right.** Similar spikes in linearized units occur immediately after a task change in convolutional networks trained with different activation functions on the CIFAR=10 dataset.

3.2 MECHANISMS OF PLASTICITY LOSS

In some cases the mechanism by which a network has lost plasticity is obvious, for example when all hidden units are saturated. In other cases, such mechanisms are harder to pin down. In this subsection, we identify two independent mechanisms by which plasticity can be lost. The first concerns distribution shifts in the preactivations, which can result in the known pathology of unit death, but also more subtle problems, such as degraded signal propagation and *linearization* of units. The second concerns the norm of the parameters of the model, which has downstream effects on the sharpness of the loss landscape and the sensitivity of the network output to changes in the weights.

Linearization and preactivation distribution shift. As discussed previously, a network consisting primarily of ‘dead’ (Lin et al., 2016) or ‘dormant’ (Sokar et al., 2023) units will not be trainable. A dead unit is unable to propagate gradients back to its incoming weights, which means that unless the unit’s input distribution serendipitously shifts in a suitable direction, the parameters associated with this unit will remain permanently frozen. (For example, a ReLU might have all negative preactivation values and thus always output 0, or a tanh might have very large preactivation values that take it into its “constant” region.) One simple solution to this problem is to use a non-saturating activation function, such as Leaky ReLU. However, changes to the mean or variance (across the training set) of the preactivation for a unit can still lead to pathologies. For example, assuming the initial preactivation distribution is chosen to achieve good signal propagation (as discussed in Section 2.1), large changes to it may harm signal propagation, leading to negative downstream effects on optimization dynamics.

Beyond harming signal propagation, a preactivation distribution change may result in more easily measurable pathologies. One such pathology is the *linearization* of a unit, whereby the unit acts as a linear (or nearly linear) transformation of its inputs, effectively reducing the expressivity of the network. For example, a ReLU unit with only positive inputs will behave like the identity, or any smooth activation unit with very low variance inputs will behave close to a linear function. Unlike dead units, through which gradients cannot flow by definition, these ‘zombie units’ can propagate gradients (and indeed permit “perfect” signal propagation); however, the presence of too many of them will reduce the *effective* expressive power of the network (e.g. Martens et al., 2013; Montufar et al., 2014; Raghu et al., 2017). While saturated units have received much attention as a factor in plasticity loss, the accumulation of effectively-linear units has not previously been studied in the context of network plasticity.

To study how training dynamics might drive such pre-activation distribution shifts immediately after a sudden change to the training objective, we train a neural network to convergence on one set of random labels of MNIST, and then re-shuffle the labels and continue optimization from the converged parameters and optimizer state on the new label set. We observe in Figure 2 two phases of training dynamics after a task change: first an ‘erasing’ phase, where confidence in incorrect predictions is rapidly reduced and predictive entropy increases, followed by a ‘disentanglement’ phase where the network adapts to fit the structure of the new task and predictive entropy declines. These two phases are exemplified with a marked increase in the number of dead units immediately after the task switch, followed by a re-equilibration period during which some units desaturate. We illustrate these two phases in the left-hand-side plot in

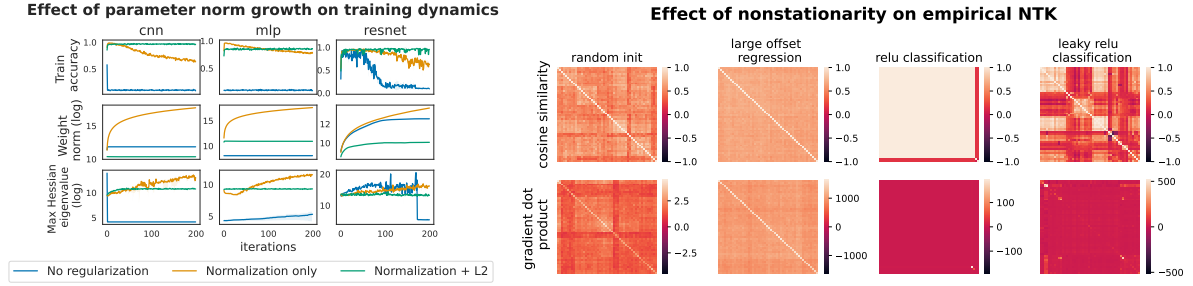


Figure 3: **Left.** Visualization of the effect of parameter scale on learning. **Right.** Visualization of the empirical NTKs of a variety of networks which exhibit varying degrees of plasticity loss, taken from previously-studied scenarios. We observe greater interference in networks which have lost plasticity than in a random initialization (i.e. greater magnitude of the cosine similarities in the top row), and also exhibit greater gradient norm variance across inputs.

Figure 2, where we track the loss and number of dead units in a small MLP immediately after a label re-randomization. We see that initially the updates on the incoming weights to the first layer tend to either increase or decrease the preactivation values for *all training inputs to the unit*. This can be explained by the pressure from the loss function in the erasing phase to reduce the magnitude of incorrect logits and increase the entropy of the predicted distribution (see Figure 20 in Appendix D for further details). Coupled with large gradient magnitudes, and hence large step sizes immediately after a task switch, this results in a high rate of units entering the linearized regime, either saturated or unsaturated, as the large updates which increase or decrease the preactivation value for all inputs quickly push these preactivations to all lie either above or below zero. As the optimization dynamics shift towards increasing the logits of correct labels, the gradient directions become more diverse, and the nonlinear structure of the network recovers.

Parameter norm growth. We observe two main learning difficulties associated with growth in the network parameters. The first is training instabilities, which can take the form of numerical overflow errors if parameter norm is allowed to grow indefinitely, but also more subtle pathologies. As we see in Figure 3, parameter norm tends to correlate with the sharpness as measured by the Hessian. Such a connection between loss landscape sharpness and last-layer parameter norm is explored in the context of edge-of-stability dynamics by Damian et al. (2022). A more subtle issue can also arise when the norms of different layers grow at different rates. Many recent works on the infinite-width limit propose scaling the learning rates for different network layers in proportion to their parameter count as a means of ensuring that updates to each layer have an equal effect on the network output (Jacot et al., 2018; Yang, 2019). However, as we demonstrate in Appendix F.2, it is not uncommon for parameters to grow unevenly through the network, leading to learning difficulties as updates to each layer have differential effects on the network output. The second concern is that increasing parameter magnitude can have the effect of saturating network components such as softmax attention heads (Wortsman et al., 2023; Dehghani et al., 2023) or normalization layers (Merrill et al., 2021), resulting in smaller changes in the network output for a fixed update step size than could be obtained at initialization.

We illustrate these two mechanisms of action in the iterated random label memorization tasks studied in the previous subsection by training a variety of architectures on iteratively re-randomized labels of CIFAR-10 for 15 million optimizer steps. We see in the leftmost plot of Figure 3 that increased parameter norm accompanies plasticity loss in all architectures we evaluate. In networks using normalization layers, the performance on new tasks degrades smoothly as the parameter norm grows. In networks which do not incorporate normalization layers, we see rapid early increases followed by plateaus due to saturated nonlinearities. Notably, there is not a monotone relationship between parameter norm at the end of training and plasticity: networks with all saturated units do not propagate gradients which could contribute to norm growth. This explains how parameter norm can be a *causal* factor in plasticity loss, while also not exhibiting a monotonic relationship.

3.3 CHARACTERIZING NETWORKS WHICH HAVE LOST PLASTICITY

Thus far, we have identified a variety of external and internal factors influencing plasticity in a neural network. We now explore whether these different paths ultimately take networks to the same end point. To do so, we look at the structure of the gradients of the network’s outputs (not its loss) with respect to its parameters using the empirical neural tangent kernel (NTK) of the network. The empirical NTK matrix (eNTK for short) is the matrix of gradient dot products between all data points in the training set, a matrix which is both expensive to compute and difficult to visualize in its full resolution. In practice, we sample a mini-batch of size 64 and compute the matrix of gradient dot products $\langle \nabla_{\theta} f(\mathbf{x}), \nabla_{\theta} f(\mathbf{x}') \rangle$ for all inputs $\mathbf{x}, \mathbf{x}'$ in the mini-batch.

We visualize these matrices as heat maps in Figure 3. To interpret the heat maps, it is helpful to think of the empirical NTK as decomposing into the sum of two matrices: a diagonal matrix D_θ and a non-diagonal matrix G_θ . Intuitively, the structure and magnitude of G_θ characterizes how the network generalizes. A network which does not generalize at all between inputs would have a diagonal eNTK, i.e. $G_\theta = 0$. In contrast, a network which multiplies the input by zero and then adds a learnable constant can be expressed as $G_\theta = c\mathbf{1}$ for some c where $\mathbf{1}$ is the all-ones matrix, and $D_\theta = 0$, and its eNTK will have rank 1. In practice, it is desirable for the matrix G_θ to encode some geometric information about the inputs, so that the network generalizes more strongly between similar inputs and weakly between distant ones, although this is not strictly necessary for trainability Xiao et al. (2020).

The eNTK characterizes the local optimization dynamics (Jacot et al., 2018) of the network, analogous to the use of the infinite-width limit NTK to identify trainable initializations (Martens et al., 2021). An ill-conditioned eNTK serves as a signal that the network is suffering from optimization difficulties in the region of parameter space it has arrived at via its training trajectory. We give a heuristic argument for why the eNTK might be of particular interest in the case of *nonstationary* learning problems as follows. First, we recall that to a first-order approximation, the change in the loss obtained by following gradient descent on a dataset $\mathbf{X} \in \mathbb{R}^{N \times d}$, $\mathbf{y} \in \mathbb{R}^N$ of size N with parameters θ , function approximator $f_\theta : \Theta \times \mathbb{R}^d \rightarrow \mathbb{R}$, and objective $\ell(\theta) = \|f(\theta, \mathbf{X}) - \mathbf{y}\|^2$ can be expressed as

$$\ell(\theta_t) - \ell(\theta_{t+1}) \approx (f(\theta_t, \mathbf{X}) - \mathbf{y})^\top K_{\theta_t}(\mathbf{X}, \mathbf{X})(f(\theta_t, \mathbf{X}) - \mathbf{y}) + \text{h.o.t.}$$

where $K_{\theta_t}(\mathbf{X}, \mathbf{X}) \in \mathbb{R}^{N \times N}$ denotes the eNTK at timestep t . Supposing that $\mathbf{y}_t$ is dependent on t , then we cannot guarantee that a value of K_{θ_t} that was previously aligned with the residual $\mathbf{y} - f(\theta_t, \mathbf{X})$ will continue to have this property as $\mathbf{y}_t$ evolves. As a result, maintaining a non-collapsed NTK is of critical importance.

We now revisit the conditions previously observed to induce training difficulties, and explore how loss of plasticity manifests in pathologies in the eNTK. We refer to Figure 3, which features two rows, the top of which visualizes the matrix of cosine similarities between gradients (i.e. it is the eNTK pre- and post-multiplied by a diagonal normalization matrix) and the bottom of which visualizes the eNTK itself. The leftmost column of this figure is a reference value for the empirical NTK of a randomly initialized network. In the second column is a network trained on a regression problem whose targets have a large mean relative to their variance. The eNTK of this network is close in ℓ_2 distance to the sum of a diagonal matrix and a rank-1 matrix, despite having only trained for a few hundred steps. The third column visualizes the eNTK of a ReLU network trained to memorize a sequence of random labels. The network has accumulated a number of dead units, resulting in a block-diagonal structure. Replacing the ReLU activation with a Leaky ReLU in the rightmost column results in a similar, although less extreme, pathology. Strikingly, despite arising from completely different learning dynamics, all three of the networks which have loss plasticity tend to exhibit similar pathologies in their eNTK, centering around the proximity of the off-diagonal entries to a low rank matrix. While it is not straightforward to explicitly regularize the eNTK of a network during training, this observation points to the utility of the eNTK as a diagnostic tool for the loss of plasticity in neural networks.

4 MITIGATION STRATEGIES

The previous section identified a handful of mechanisms by which neural networks can lose plasticity: roughly speaking, these centered around growth of the parameter norm, distribution shift in the preactivations (leading to saturated, dead, and zombie units, among other possible pathologies associated with poor signal propagation), and poor conditioning of the features due to training with large regression target means. In this section, we will investigate a variety of means by which these mechanisms can be intervened on to prevent plasticity loss. We then explore whether it is possible to combine the interventions for these independent mechanisms to achieve an additive benefit. For full details of the experiment settings in this section, we refer to Appendix C.

4.1 ADDRESSING INDIVIDUAL MECHANISMS

Unbounded norm growth. To prevent feature and parameter norms from growing indefinitely, one can either enforce hard normalization constraints or use softer regularization strategies. We evaluate a variety of these in the leftmost subplot in Figure 4. We find that normalizing features along either the layer or batch dimension does not adversely affect network expressivity or learning speed; however, constraining the norm of the weights (in our case, by rescaling the per-layer vectorized weight norm to its value at initialization, a heuristic proxy to path normalization (Neyshabur et al., 2016)) did interfere with optimization speed. Conversely, regularizing the norm of the features was less effective at combating plasticity loss than applying the hard constraint of layer or batch normalization.

Preactivation statistics. One natural solution to the problem of dead units is to simply not use saturating activation functions (Abbas et al., 2023); however, this strategy does not resolve the underlying problem of preactivation

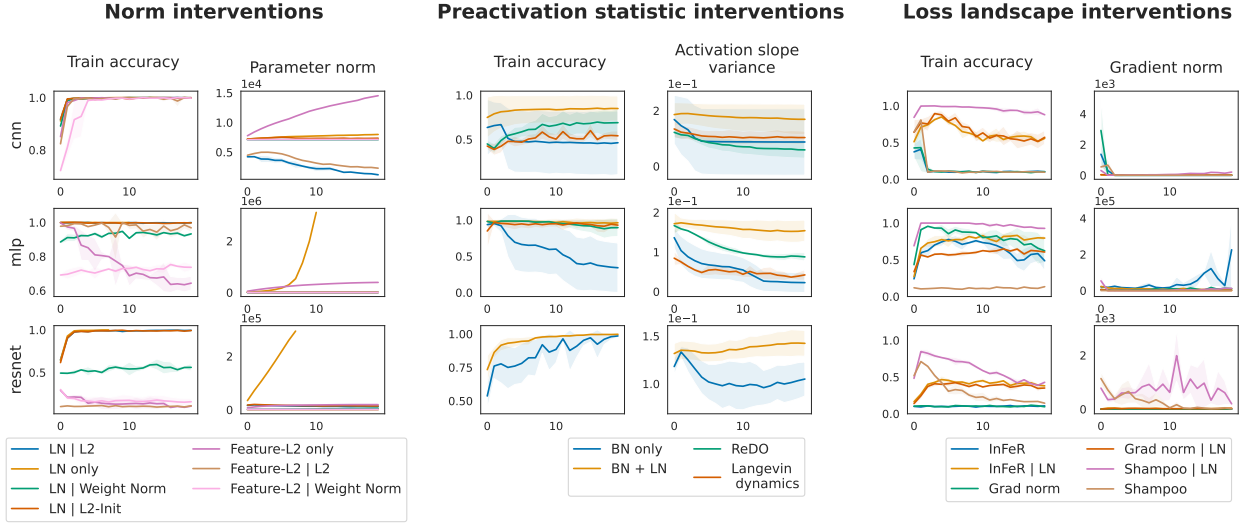


Figure 4: Comparison of interventions aimed at addressing different failure modes: overall, combining layer normalization with L2 regularization addressed plasticity loss in all classification problems we considered. Many other strategies also improve over doing nothing, but do not outperform this baseline.

distribution shift. We consider two primary classes of interventions: constraining the pre-activations to have zero mean and unit variance along either the batch or layer dimension (using batch normalization and layer normalization respectively), and resetting units which have saturated using ReDO (Sokar et al., 2023). We observe in the central subplot of Figure 4 that normalizing the preactivations has beneficial effects on both plasticity in later task changes and on convergence rates in each task. In contrast, methods which reset dead or colinear features are more robust to plasticity loss than standard training approaches, but sometimes slow down convergence on single tasks and are unable to resolve signal propagation issues in larger networks that arise at initialization, which is why they are not included in our evaluations on ResNets. We include a more detailed sweep over more creative normalization layer combinations in Figure 9 of Appendix F, finding slight improvements over the standard options considered here.

Loss landscape conditioning. The regularization and normalization strategies studied previously occurred on the level of maintaining the expressivity of individual units. However, it is plausible that this unit-level intervention misses out on global properties of the loss landscape which may also benefit from regularization. We test this hypothesis in the rightmost subplot in Figure 4, where we consider a squared gradient norm penalty (Barrett & Dherin, 2020; Smith et al., 2021), regularization of a feature subspace towards its initial value (InFeR, Lyle et al., 2021), and the curvature-aware optimizer Shampoo (Gupta et al., 2018; Anil et al., 2020). All results are run in networks with layer normalization but without L2 regularization. While we see some performance improvements at the end of training on some nonstationary tasks, none of these approaches consistently outperforms the combination of L2 regularization and layer normalization (of the pre-activations) studied earlier.

Target scale. Having observed benefits of layer normalization and L2 regularization in classification problems, we now turn our attention to regression, taking a small digression to revisit a reinforcement learning problem where plasticity loss was previously identified (Lyle et al., 2023). In this environment, the agent is randomly shown an observation from an image classification dataset and a reward of 1 is given if the action taken by the agent corresponds to the label of the image it observes. The resulting optimal value prediction problem amounts to predicting a one-hot vector corresponding to the class label of an image plus a constant bias term (whose value converges to roughly $(1 - \gamma)^{-1}$). We train a DQN agent (see Appendix C.3 for details) on this task, and evaluate the plasticity of parameter checkpoints by regressing from those parameters to a fixed-norm perturbation of the network’s current outputs. As before, the large target norms induced by the high discount factor result in plasticity loss even in the network incorporating layer normalization.

Distributional losses (Bellemare et al., 2017; Imani & White, 2018; Schrittwieser et al., 2020; Stewart et al., 2023) have been demonstrated to improve performance on a variety of regression problems, and have been found to help mitigate plasticity loss (Lyle et al., 2023). In particular, we make use of the ‘two-hot’ trick (described in full detail in Appendix C.3), which encodes the regression problem as a classification objective, by placing a grid of points over the range of possible output values, and for each real-valued target, putting probability mass on the two closest points (proportional to its proximity to them). The cross-entropy loss between the predicted and target distribution is then

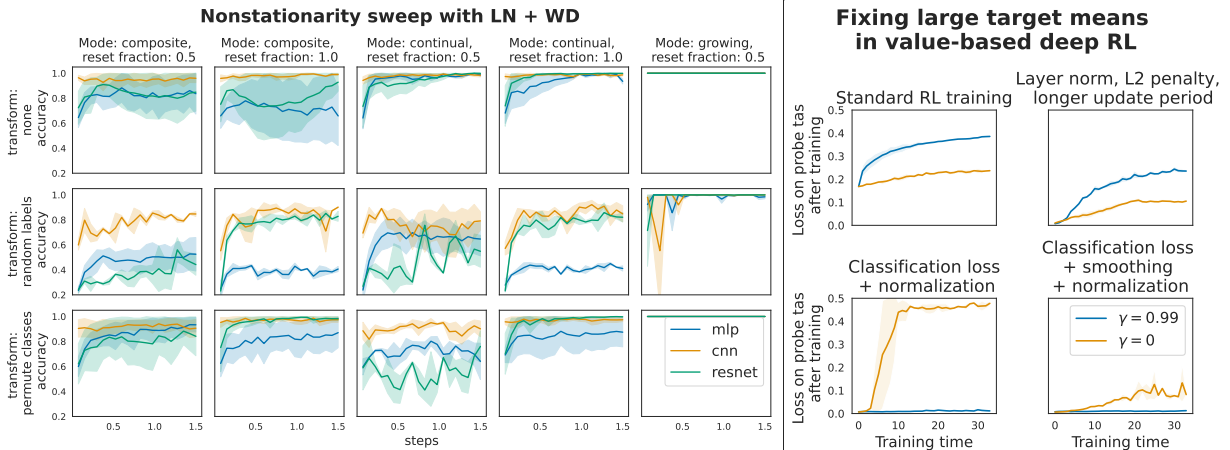


Figure 5: **Left.** Layer normalization and L2 regularization on synthetic non-stationary supervised classification problems. **Right.** necessity of scale-invariant output parameterization in a simple RL task.

used to replace the L2 distance between the means of these distributions. We see in the bottom left subplot of the right side of Figure 5 that this appears to completely mitigate the problem for the $\gamma = 0.99$ agent. The failure of $\gamma = 0$ in the case of classification losses illustrates a more subtle issue which stems from the saturated softmax outputs, and consequent loss spike obtained when we evaluate the network on a new task, which we previously discussed in Section 3.2. This non-smoothness can be mediated by label smoothing (bottom right subplot) where the target distribution (defining the cross-entropy loss) is a mixture of a uniform distribution and the original two-hot encoding of the real-valued regression targets. This trick has the effect of scaling the mean of the target distribution by a constant factor, but because it does so uniformly for all inputs it does not interfere with accurate ranking and is easy to correct for if desired. Our findings here suggest an intriguing mechanism for some of the benefits of categorical distributional reinforcement learning: not only does the distributional loss provide smoother gradients (Imani & White, 2018), but it also allows the network to fit large scalar values without developing ill-conditioned feature representations.

4.2 EVALUATION

The previous subsection observed that the combination of weight decay and layer normalization was highly effective at mitigating plasticity loss. This is in line with our previous discussion of the mechanisms on which these interventions act: layer normalization benefits the preactivation distribution, while weight decay avoids catastrophic growth of the weight norm. We now explore how this combination fares under a broader range of nonstationarities.

Supervised learning. Consistently throughout our preceding evaluations, we have observed significant benefits from the simple combination of layer normalization and L2 regularization. We now expand our evaluations to incorporate input distribution shifts, gradually expanding datasets, and composite forms of nonstationarity where only a subset of inputs change over time. In each of the modes, re-randomization of the labels happens at a regular interval of 75k iterations, partitioning the training into a sequence of tasks. We evaluate three architectures: a MLP, a CNN, and a ResNet-18 as detailed previously. Each architecture has layer normalization applied before each nonlinearity, and is trained with a fixed L2 penalty of 10^{-5} (additional tuning of the L2 penalty can yield better per-task performance, as can be seen in Figure 22 in the Appendix). We use the CIFAR-10 benchmark as our base dataset. Each network is trained for 1.5M optimizer steps, which corresponds to 20 tasks. In the ‘continual’ mode, a random transformation is applied to a given fraction of the dataset (chosen uniformly at random), either 0.5 or 1.0. In the ‘composite’ mode, the random transformation is always applied to the same data points. In the ‘growing’ mode, we start with inputs from a single class, and add inputs from an additional class at each task boundary, also applying random transformations to the data incorporated so far. Encouragingly, as the network trains on the sequence of 20 tasks we tend to see improving accuracy over time (left side of Figure 5), suggesting that provided mild guard rails are put in place, several pathologies can be avoided while the network learns a useful inductive bias for the underlying task distribution.

Reinforcement learning. Improving performance in RL requires striking a delicate balance between maintaining plasticity and not interfering with other aspects of the algorithm. Some regularization methods, such as L2 penalties and batch normalization, are known to interfere with learning in deep RL (Salimans & Kingma, 2016), constraining the set of reasonable interventions to consider. We therefore focus on applying layer normalization to the preactivations of a C51 and a Rainbow agent trained on the arcade learning environment. These agents combine both scale-invariant output encoding and the normalization approach discussed in Section 4. We see in Figure 6 that this approach does

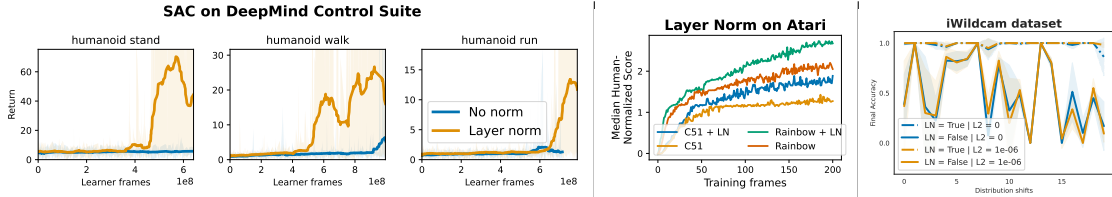


Figure 6: Left: performance of a soft actor-critic agent on the humanoid domain from the DeepMind Control Suite using architectures which include or exclude normalization layers. Middle: effect of incorporating layer normalization on value-based RL agents trained on all 57 games of the Arcade Learning Environment benchmark. Right: effect of normalization and L2 regularization on robustness to changes in photo location on the iWildcam dataset.

consistently provide modest performance improvements over equivalent baselines. We found that batch normalization and L2 regularization interfere with learning; these experiments and per-game performance of all agents can be found in Appendix F.4. We also validate the beneficial effects of layer normalization on a Soft Actor-Critic agent trained on the DeepMind Control suite. We use fully connected architectures, applying layer normalization prior to each nonlinearity as in the Atari networks, with full details described in Appendix C.2. Here again we find that layer normalization provides an almost uniform improvement over the suite. We see particularly striking improvements in the challenging humanoid domain as seen in Figure 6. Results for the full suite can be found in Appendix F.11.

Natural distribution shifts. We further verify the robustness of our approach to natural distribution shifts using a dataset from the WiLDS benchmark (Koh et al., 2021) which consists of nature camera photos taken from different locations and from different times of day. We follow an iterative training procedure, where we sample a location and train on images taken in that location for 2000 optimizer steps, then sample a new location and continue training on images from this location. We repeat this process 20 times, averaging over 5 random seeds. Similarly to the synthetic tasks, we see in Figure 6 that networks trained with L2 regularization and preactivation normalization are better able to deal with changes in the training data distribution, obtaining lower loss on later training sets than networks trained without these interventions. Although performance in the unregularized networks does not monotonically decline due to variable difficulty of different locations, we observe a widening gap between the final loss attained by the networks which use layer normalization and L2 regularization, confirming the efficacy of these interventions. A more detailed set of analyses can be found in Figure 24.

5 CONCLUSIONS

This paper has shown that while no single network property can explain all instances of plasticity loss in neural networks, a handful of independent mechanisms are responsible for a large fraction of observed cases. Some of these mechanisms are well-known to the community, such as dead ReLU units, but others such as large target offsets and preactivation distribution shift (resulting in unit “zombification”, among other problems) had not been previously identified and present exciting directions for future study, both as mechanisms of plasticity loss, and as potential targets for its remedies. We have further shown that by identifying effective mitigation strategies for each mechanism in isolation and then combining the most effective interventions, we can significantly reduce the combinatorial complexity of the search space of multi-component interventions. We anticipate that this divide-and-conquer strategy may prove useful to future works which aim to find even better mitigation strategies than those identified here. In particular, we note that while L2 regularization is effective at avoiding extreme weight norms (which can interfere with learning), it does so at the expense of convergence speed, and it’s possible that better norm control strategies exist which do not interfere as much with single-task training speed.

It is useful to place these findings in the broader literature on continual learning, and in particular within the framework of stability-plasticity trade-offs. The pathologies studied in this paper generally reduce both stability and plasticity. For example, while a network filled with dead units might be ‘stable’ in the sense of not changing very much, it will clearly be unable to yield any useful information about its past training data. Thus, existing training methodologies which are vulnerable to these pathologies are still far from the Pareto frontier of stability-plasticity trade-offs. We are hopeful that further work will be able to use our observations as a stepping stone towards developing neural networks which can fully utilize their capacity in non-stationary learning problems to both maintain information about the past and adapt to future changes. It is likely that further work studying more subtle forms of plasticity loss, and in particular negative transfer between early and later training data, may identify additional mechanisms for which this trade-off is more pronounced. We expect that many more subtle presentations plasticity loss which occur in larger-scale systems will be of this flavour, and believe that the style of analysis presented in this work is an exciting avenue to yield practical insights into the training of these systems.

REFERENCES

- Zaheer Abbas, Rosie Zhao, Joseph Modayil, Adam White, and Marlos C Machado. Loss of plasticity in continual deep reinforcement learning. *ICML*, 2023.
- Rohan Anil, Vineet Gupta, Tomer Koren, Kevin Regan, and Yoram Singer. Scalable second order optimization for deep learning. *arXiv preprint arXiv:2002.09018*, 2020.
- Kavosh Asadi, Rasool Fakoor, and Shoham Sabach. Resetting the optimizer in deep rl: An empirical study. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- Jordan Ash and Ryan P Adams. On warm-starting neural network training. *Advances in Neural Information Processing Systems*, 33:3884–3894, 2020.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- David Balduzzi, Marcus Frean, Lennox Leary, JP Lewis, Kurt Wan-Duo Ma, and Brian McWilliams. The shattered gradients problem: If resnets are the answer, then what is the question? In *International Conference on Machine Learning*, pp. 342–350. PMLR, 2017.
- David GT Barrett and Benoit Dherin. Implicit gradient regularization. *arXiv preprint arXiv:2009.11162*, 2020.
- Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning*, pp. 449–458. PMLR, 2017.
- Tudor Berariu, Wojciech Czarnecki, Soham De, Jorg Bornschein, Samuel Smith, Razvan Pascanu, and Claudia Clopath. A study on the plasticity of neural networks. *arXiv preprint arXiv:2106.00042*, 2021.
- Alex Damian, Eshaan Nichani, and Jason D. Lee. Self-stabilization: The implicit bias of gradient descent at the edge of stability. In *OPT 2022: Optimization for Machine Learning (NeurIPS 2022 Workshop)*, 2022. URL https://openreview.net/forum?id=enoU_Kp7Dz.
- Amit Daniely, Roy Frostig, and Yoram Singer. Toward deeper understanding of neural networks: The power of initialization and a dual view on expressivity. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *International Conference on Machine Learning*, pp. 7480–7512. PMLR, 2023.
- Shibhansh Dohare, A Rupam Mahmood, and Richard S Sutton. Continual backprop: Stochastic gradient descent with persistent randomness. *arXiv preprint arXiv:2108.06325*, 2021.
- Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *International Conference on Machine Learning*, pp. 1842–1850. PMLR, 2018.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. PMLR, 2018.
- Boris Hanin and Mihai Nica. Finite depth and width corrections to the neural tangent kernel. *arXiv preprint arXiv:1909.05989*, 2019.
- Soufiane Hayou, Arnaud Doucet, and Judith Rousseau. On the impact of the activation function on deep neural networks training. In *International conference on machine learning*, pp. 2672–2680. PMLR, 2019.
- Soufiane Hayou, Eugenio Clerico, Bobby He, George Deligiannidis, Arnaud Doucet, and Judith Rousseau. Stable resnet. In *International Conference on Artificial Intelligence and Statistics*, pp. 1324–1332. PMLR, 2021.
- Bobby He, James Martens, Guodong Zhang, Aleksandar Botev, Andrew Brock, Samuel L Smith, and Yee Whye Teh. Deep transformers without shortcuts: Modifying self-attention for faithful signal propagation. *arXiv preprint arXiv:2302.10322*, 2023.

- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Maximilian Igl, Gregory Farquhar, Jelena Luketina, Wendelin Boehmer, and Shimon Whiteson. Transient non-stationarity and generalisation in deep reinforcement learning. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=Qun8fv4qSby>.
- Ehsan Imani and Martha White. Improving regression performance with distributional losses. In *International conference on machine learning*, pp. 2157–2166. PMLR, 2018.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pp. 448–456. pmlr, 2015.
- Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems*, 31, 2018.
- Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, Michihiro Yasunaga, Richard Lanus Phillips, Irena Gao, et al. Wilds: A benchmark of in-the-wild distribution shifts. In *International Conference on Machine Learning*, pp. 5637–5664. PMLR, 2021.
- Aviral Kumar, Rishabh Agarwal, Dibya Ghosh, and Sergey Levine. Implicit under-parameterization inhibits data-efficient deep reinforcement learning. In *International Conference on Learning Representations*, 2020.
- Saurabh Kumar, Henrik Marklund, Ashish Rao, Yifan Zhu, Hong Jun Jeon, Yueyang Liu, and Benjamin Van Roy. Continual learning as computationally constrained reinforcement learning. *arXiv preprint arXiv:2307.04345*, 2023a.
- Saurabh Kumar, Henrik Marklund, and Benjamin Van Roy. Maintaining plasticity via regenerative regularization. *arXiv preprint arXiv:2308.11958*, 2023b.
- Hoon Lee, Hanseul Cho, Hyunseung Kim, Daehoon Gwak, Joonkee Kim, Jaegul Choo, Se-Young Yun, and Chulhee Yun. Plastic: Improving input and label plasticity for sample efficient reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. Deep neural networks as gaussian processes. *arXiv preprint arXiv:1711.00165*, 2017.
- Alex Lewandowski, Haruto Tanaka, Dale Schuurmans, and Marlos C Machado. Curvature explains loss of plasticity. *arXiv preprint arXiv:2312.00246*, 2023.
- Zhouhan Lin, Roland Memisevic, and Kishore Konda. How far can we go without convolution: Improving fully-connected networks. *ICLR Workshop Track*, 2016.
- Clare Lyle, Mark Rowland, and Will Dabney. Understanding and preventing capacity loss in reinforcement learning. In *International Conference on Learning Representations*, 2021.
- Clare Lyle, Mark Rowland, Will Dabney, Marta Kwiatkowska, and Yarin Gal. Learning dynamics and generalization in deep reinforcement learning. In *International Conference on Machine Learning*, pp. 14560–14581. PMLR, 2022.
- Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Understanding plasticity in neural networks. In *International Conference on Machine Learning*, 2023.
- James Martens, Arkadev Chattopadhyay, Toni Pitassi, and Richard Zemel. On the representational efficiency of restricted boltzmann machines. *Advances in Neural Information Processing Systems*, 26, 2013.
- James Martens, Andy Ballard, Guillaume Desjardins, Grzegorz Swirszcz, Valentin Dalibard, Jascha Sohl-Dickstein, and Samuel S Schoenholz. Rapid training of deep neural networks without skip connections or normalization layers using deep kernel shaping. *arXiv preprint arXiv:2110.01765*, 2021.
- William Merrill, Vivek Ramanujan, Yoav Goldberg, Roy Schwartz, and Noah A Smith. Effects of parameter norm growth during transformer training: Inductive bias from gradient descent. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 1766–1781, 2021.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

- Guido Montufar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. *NeurIPS*, 2014.
- Behnam Neyshabur, Yuhuai Wu, Russ R Salakhutdinov, and Nati Srebro. Path-normalized optimization of recurrent neural networks with relu activations. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/file/74563ba21a90da13dacf2a73e3ddefa7-Paper.pdf.
- Evgenii Nikishin, Max Schwarzer, Pierluca D’Oro, Pierre-Luc Bacon, and Aaron Courville. The primacy bias in deep reinforcement learning. In *International Conference on Machine Learning*, pp. 16828–16847. PMLR, 2022.
- Evgenii Nikishin, Junhyuk Oh, Georg Ostrovski, Clare Lyle, Razvan Pascanu, Will Dabney, and André Barreto. Deep reinforcement learning with plasticity injection. *International Conference on Learning Representations*, 2023.
- Mohammad Pezeshki, Oumar Kaba, Yoshua Bengio, Aaron C Courville, Doina Precup, and Guillaume Lajoie. Gradient starvation: A learning proclivity in neural networks. *Advances in Neural Information Processing Systems*, 34: 1256–1272, 2021.
- Ben Poole, Subhaneil Lahiri, Maithra Raghu, Jascha Sohl-Dickstein, and Surya Ganguli. Exponential expressivity in deep neural networks through transient chaos. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL <https://proceedings.neurips.cc/paper/2016/file/148510031349642de5ca0c544f31b2ef-Paper.pdf>.
- Maithra Raghu, Ben Poole, Jon Kleinberg, Surya Ganguli, and Jascha Sohl-Dickstein. On the expressive power of deep neural networks. In *international conference on machine learning*, pp. 2847–2854. PMLR, 2017.
- Roberta Raileanu and Rob Fergus. Decoupling value and policy for generalization in reinforcement learning. In *International Conference on Machine Learning*, pp. 8787–8798. PMLR, 2021.
- Tim Salimans and Durk P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. *Advances in neural information processing systems*, 29, 2016.
- Samuel S. Schoenholz, Justin Gilmer, Surya Ganguli, and Jascha Sohl-Dickstein. Deep information propagation. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=H1W1UN9gg>.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.
- Samuel L Smith, Benoit Dherin, David GT Barrett, and Soham De. On the origin of implicit regularization in stochastic gradient descent. *arXiv preprint arXiv:2101.12176*, 2021.
- Ghada Sokar, Rishabh Agarwal, Pablo Samuel Castro, and Utku Evci. The dormant neuron phenomenon in deep reinforcement learning. *ICML*, 2023.
- Lawrence Stewart, Francis Bach, Quentin Berthet, and Jean-Philippe Vert. Regression as classification: Influence of task formulation on neural network features. In *International Conference on Artificial Intelligence and Statistics*, pp. 11563–11582. PMLR, 2023.
- Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *International conference on machine learning*, pp. 1139–1147. PMLR, 2013.
- Hado P van Hasselt, Arthur Guez, Matteo Hessel, Volodymyr Mnih, and David Silver. Learning values across many orders of magnitude. *Advances in neural information processing systems*, 29, 2016.
- Mitchell Wortsman, Peter J Liu, Lechao Xiao, Katie Everett, Alex Alemi, Ben Adlam, John D Co-Reyes, Izzeddin Gur, Abhishek Kumar, Roman Novak, et al. Small-scale proxies for large-scale transformer training instabilities. *arXiv preprint arXiv:2309.14322*, 2023.
- Lechao Xiao, Yasaman Bahri, Jascha Sohl-Dickstein, Samuel Schoenholz, and Jeffrey Pennington. Dynamical isometry and a mean field theory of cnns: How to train 10,000-layer vanilla convolutional neural networks. In *International Conference on Machine Learning*, pp. 5393–5402. PMLR, 2018.

Lechao Xiao, Jeffrey Pennington, and Samuel Schoenholz. Disentangling trainability and generalization in deep neural networks. In *International Conference on Machine Learning*, pp. 10462–10472. PMLR, 2020.

Jingjing Xu, Xu Sun, Zhiyuan Zhang, Guangxiang Zhao, and Junyang Lin. Understanding and improving layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

Greg Yang. Wide feedforward or recurrent neural networks of any architecture are gaussian processes. *Advances in Neural Information Processing Systems*, 32, 2019.

Guodong Zhang, Aleksandar Botev, and James Martens. Deep learning without shortcuts: Shaping the kernel with tailored rectifiers. In *International Conference on Learning Representations*, 2021.

A APPENDIX

B ADDITIONAL BACKGROUND

B.1 NEURAL NETWORK TRAINING

Neural networks compute a sequence of feature vectors – whose entries are called units – through a sequence of transformations called layers. Common types of layers include affine/linear ones, that typically multiply by a weight matrix and add a bias vector, nonlinear layers, which typically apply an element-wise nonlinear function to their input (called an activation function), such as ReLU or tanh. Also increasingly common, and important to this work, are normalization layers, which normalize their inputs so that either 1) each entry/unit of the vector has sample mean 0 and variance 1 over the current minibatch of training data (called batch normalization (Ioffe & Szegedy, 2015)), or 2) the vector has sample mean 0 and variance 1 across its entries/units (called layer normalization (Ba et al., 2016)).

The weights of the neural networks are initialized to random small values, typically sampled from a zero-mean Gaussian, with two goals in mind: 1) to break symmetries in the network, allowing different hidden units to represent different features, and 2) to allow gradients to preserve norm, typically in expectations, making certain assumptions about the activation function in order to reason about the non-linearity of the system. Training is carried out by randomly sampling initial values of the parameters (typically the weights and biases of the affine layers) and then iteratively optimizing them with respect to an objective function. Objective functions are usually defined as the expected loss over the training set (with often additional regularization terms), and are estimated via iid sampling from the training set for the purposes of stochastic optimization. The most commonly used type of regularization is L2 regularization or Weight Decay (WD), which penalizes the squared norm of the parameter vector. Weight decay can also be incorporated directly into the weight update rule as opposed to being specified in the objective function to encourage the weights to be small in magnitude. The interplay between WD and BN has been extensively studied for a single task setting but remains poorly understood in the context of non-stationary data distribution.

Furthermore, we are specifically interested in studying the effects of the distribution shifts on the network’s ability to retain the ability to continually learn. Due to the complexities of the optimization process, various factors contribute to how learning evolves especially in the presence of non-stationarity. In this work, we focus on the following phenomena: 1) *unit saturation* (also referred to as gradient starvation (Pezeshki et al., 2021)) measures the sufficiency of a gradient signal, the lack of which can prevent necessary features from being learned, 2) *pre-activation distribution shift* measures the shift in the pre-activations of a unit which are often initialized to be constrained to zero-mean Gaussian as discussed before, 3) *parameter growth* is aimed at measuring the growth in the magnitude of the norms of the parameters and features as large magnitudes can lead to instabilities in the optimization process, we use the notion of effective dimensionality of a network’s representation to quantify this, and finally 4) *loss landscape pathologies* are geared towards quantifying the sensitivity of the network’s outputs to changes in its parameters, we use the notion of effective feature rank and its interplay with gradient similarity to measure this formally. We defer the formal details of how these are calculated to the respective sections in the paper later.

C EXPERIMENT DETAILS

C.1 NETWORK ARCHITECTURES

We consider three classes of network architecture: a fully-connected multilayer perceptron (MLP), for which we default to a width of 512 and depth of 4 in our evaluations; a convolutional network with k convolutional layers

followed by two fully connected layers, for which we default to depth four, 32 channels, and fully-connected hidden layer width of 256; we also consider a ResNet-18 architecture, which follows the standard architecture.

In all networks, we apply layer normalization before batch normalization if both are used at the same time. By default, we typically use layer normalization rather than batch normalization, although in many ablations we consider various combinations of one or both.

C.2 CONTINUAL SUPERVISED LEARNING

Our continual supervised learning domain is constructed from a fixed image classification dataset: we have considered CIFAR-100, CIFAR-10, and MNIST and observe consistent results across all base datasets. We primarily use CIFAR-10 in our evaluations. Each continual classification problem is characterized by an input transformation and a label transformation. For input transformations, we use the identity transformation permutation of the image pixels. For label transformations, we permute classes (for example, all images with the label 5 will be re-assigned the label 2), and random label assignment, where each input is uniformly at random assigned a new label independent of its class in the underlying classification dataset.

We fix an iteration interval of 75000 optimizer steps, during which the network is trained using the adam optimizer. This number was selected as it was sufficient for all architectures to converge on most data transformations. We conducted a sweep over learning rates for the different architectures, settling on $1e-5$ for the resnet, $1e-4$ for the convolutional network, and $1e-5$ for the MLP, to ensure that all networks could at least solve the single-task version of each label and target transformation. We then alternate between training the network, and applying a new random transformation to the dataset, for a fixed number of iterations, typically 20.

C.3 CONTEXTUAL BANDITS

In the contextual bandit tasks, we train a reinforcement learning agent on a stationary environment using Q-learning with target networks as in DQN (Mnih et al., 2015). The environment is based on an image classification dataset (we consider CIFAR-10 in this paper), and is defined as an MDP with observations equal to the images in the dataset, actions equal to the number of classes, and a reward function which yields a value of α if the action taken in the state is equal to its label, and zero otherwise. The state then randomly transitions to a new image from the dataset. While it is not necessary to deploy a reinforcement learning algorithm in order to maximize the reward in this task, it provides a simple setting in which to study the loss of plasticity in reinforcement learning.

The DQN agents we train use target network update periods of either 500 or 5000 steps. We use a replay buffer of size 100,000, and have the agent follow a uniform random policy. In section 3.2 we visualize results for convolutional neural networks, but we also run experiments on ResNets and MLPs. To evaluate plasticity in these agents, we take a network checkpoint, initialize a fresh optimizer, and train the network to fit a perturbation of its initial outputs on a subset of inputs from its replay buffer. Concretely, this means that if we let θ_T denote the parameter checkpoint we start from, the objective function of the probe task at step k of the fine-tuning phase on some set of inputs X will be equal to $\|f(\theta_T; X) - f(\theta_k; X) + \eta(X)\|^2$, where η is a high-frequency function of the input X . We run this optimization procedure for a fixed number of steps (2000, selected as this was sufficient for most networks to solve the probe task at a random initialization). This probe task can be thought of as measuring the expected reduction in TD error if the network were to fit its Bellman targets with a random reward function, given some uniform prior over rewards.

The ‘two-hot’ trick: we re-parameterize regression problems as classification problems by first setting some integer bound M on the range of possible values our regression targets might take (for example, in a RL task with $\gamma = 0.9$ and maximum reward 1, we know that the maximum value of a state-action pair will be $M = 10$). We then discretize the interval $[-M, M]$ to obtain a set of atoms $\{-M, -M + 1, \dots, M - 1, M\}$. The network’s output is a $2M + 1$ -dimensional vector of logits to a softmax distribution on this set. Given a regression target c , we construct a probability distribution with support on $\lfloor c \rfloor$ and $\lceil c \rceil$, with probabilities $P(\lfloor c \rfloor) = \lceil c \rceil - c$, and $P(\lceil c \rceil) = 1 - P(\lfloor c \rfloor)$. We then minimize the cross-entropy loss between this distribution and the softmax distribution output by the network.

C.4 NATURAL NON-STATIONARITIES

WiLDS: The WiLDS benchmark consists of a collection of datasets which can be partitioned to induce distribution shifts. In keeping with prior analyses of image classification, the dataset used in Figure 6 is the *iwildcam* dataset, which consists of photos of animals taken in a variety of locations and times of day. To generate a series of distribution shifts, we train a network only on data collected from a single location; at fixed intervals, we change the location used



Figure 7: Sample from the iwildcam dataset.

to generate the training data, and continue training on this new location. We use 10 locations and repeat this process 20 times.

Arcade learning environment: we also study deep reinforcement learning agents on the arcade learning environment. While we only consider single-environment tasks, we note that the RL process itself introduces a number of rich forms of nonstationarity, in both the distribution of inputs and the learning targets. We study networks trained with the C51 algorithm, a distributional approach which approximates the probability distribution of returns from a given state-action pair, rather than their expected value, using a categorical distribution on 51 atoms. The learner then minimizes the KL divergence between its predicted distribution and the *distributional Bellman target*. The precise details of this algorithm can be found in [Bellemare et al. \(2017\)](#).

Its relevance for us is that, similarly to the two-hot trick, it leverages the scale-invariant cross-entropy loss in order to solve what is ultimately a regression problem. This allows us to verify that, as in the contextual bandits discussed previously, combining layer normalization and scale-invariant losses can allow the network to maintain its ability to adapt to changes in its prediction targets. We further note that, unlike the two-hot loss, distributional objectives are less likely to struggle with saturated softmax outputs that proved to be a challenge in the contextual bandit tasks because the distributional Bellman targets will typically exhibit nontrivial entropy.

D NEURAL TANGENT KERNEL FOR LINEARIZED NETWORKS

We provide some additional insight into why linearization can lead to pathological learning dynamics by considering the empirical neural tangent kernel obtained by a linear network. We first note that in our case a *linearized* network will have identical local dynamics to a *linear* network of the form $\prod_{l=1}^L M_\phi^l W^l$, where M_ϕ^l denotes the matrix of slopes of the nonlinearity ϕ for each feature. In a ReLU network with all positive slopes, $M_\phi^l = Id$ for each layer l , but in a network with dormant neurons one would have some zero entries on the diagonal of M_ϕ^l . Similarly, a leaky ReLU network would have some slope values of $-\alpha$, where $-\alpha$ denotes the slope assigned to negative preactivation values. Thus in general, we have

$$f_\theta(\mathbf{x}) = \left(\prod_{l=1}^L M_\phi^l W^l \right) \mathbf{x}. \quad (1)$$

For simplicity, we assume $f_\theta : \mathbb{R}^{d_{in}} \rightarrow \mathbb{R}$, with $\theta = \oplus_{k=1}^L \text{vec}(W_k)$ (i.e. there are no bias terms in the network). This results in the following per-layer Jacobian:

$$\nabla_{W^k} f_\theta(\mathbf{x}) = \prod_{l=k+1}^L M_\phi^l W^l \mathbf{x}^\top \left(\prod_{l=1}^{k-1} M_\phi^l W^l \right)^\top \quad (2)$$

If we vectorize W^k , we then obtain the following linear function of $\mathbf{x}$:

$$\begin{aligned} \text{vec}(\nabla_{W^k} f_\theta(\mathbf{x}))_{ij} &= \prod_{l=k+1}^L M_\phi^l W^l \mathbf{x}^\top \left(\prod_{l=1}^{k-1} M_\phi^l W^l \right)^\top [i, j] \\ &= \left(\prod_{l=k+1}^L M_\phi^l W^l \right) [i] \left(\prod_{l=1}^{k-1} W^l \mathbf{x} \right) [j] \end{aligned}$$

letting $M_{>k} = \prod_{l=k+1}^L M_{\phi} W^l$

$$= M_{>k}[i] \sum_{i'=1}^{d_{in}} \left(\prod_{l=1}^{k-1} M_{\phi} W^l \right) [j, i'] \mathbf{x}_{i'}$$

and analogously $M_{<k} = \prod_{l=1}^{k-1} M_{\phi} W^l$

$$\begin{aligned} &= M_{>k}[i] \sum_{i'=1}^{d_{in}} M_{<k}[j, i'] \mathbf{x}_{i'} \\ \implies \text{vec}(\nabla_{W^k} f_{\theta}(\mathbf{x})) &= A_k \mathbf{x} \text{ where } A_k[i, j] = M_{>k}[i] M_{<k}[j, i'] \end{aligned}$$

We can then write up the empirical neural tangent kernel for the entire set of parameters $\theta = W_1 \oplus \dots \oplus W_L$ as

$$\begin{aligned} K_{\theta}(\mathbf{x}, \mathbf{y}) &= \sum_{k=1}^L \langle \text{vec}(\nabla_{W^k}(f_{\theta})(\mathbf{x})), \text{vec}(\nabla_{W^k}(f_{\theta})(\mathbf{y})) \rangle \\ &= \sum_{k=1}^L \mathbf{x}^{\top} (A_k^{\top} A_k) \mathbf{y} = \mathbf{x}^{\top} \left(\sum_{k=1}^L A_k^{\top} A_k \right) \mathbf{y} \end{aligned}$$

and consequently

$$\begin{aligned} K_{\theta}(\mathbf{X}, \mathbf{X}) &= \mathbf{X}^{\top} \left(\sum_{k=1}^L A_k^{\top} A_k \right) \mathbf{X} \\ \implies \text{rank}(K_{\theta}(\mathbf{X}, \mathbf{X})) &\leq \text{rk}(\mathbf{X}) \end{aligned}$$

As a result, we see that the local training dynamics of a linearized network have bounded rank, in the best case equal to the input dimension but possibly much lower, particularly if the training samples lie in a low-dimensional subspace of the input space. For example, if a network has many dead ReLU units then it may come to be that the matrices A_k end up bottlenecking the rank of the NTK more so than the dimension of the training samples $\mathbf{X}$.

E ADDITIONAL ANALYSIS

In this section, we include additional ablations and finer-grained analyses relating to the results in the main paper.

E.1 VISUALIZING LEARNING CURVES

While most of the figures in this paper only show accuracy at the end of a task iteration, we also visualize what the loss curves look like at a more fine-grained level in Figure 8. Note that in many cases, declines in plasticity later in training often correspond to learning curves becoming shallower, rather than to higher plateaus.

E.2 NORMALIZATION LAYER COMPONENTS

In Figure 9 we ablate the roles of the different components of normalization layers. In particular we are interested in disentangling the relative importance of centering the preactivation distribution about zero, and rescaling to unit standard deviation. We find that most of the performance gain from normalization layers can be attributed to the second mechanism, although the relative effect sizes of each does vary across normalization dimension, network architecture, and dataset.

E.3 INTERACTION BETWEEN NONLINEARITY AND NORMALIZATION

In Figure 10, we evaluate the importance of normalization layers centering their normalization precisely around the region where most nonlinearities exhibit the greatest effect (i.e. zero in the case of ReLU, GeLU, tanh, and leaky ReLU networks). In particular, we see whether adding normalization layers still exhibits a similar benefit to plasticity when the normalization centering occurs away from the ‘receptive field’ of the nonlinearity. We study this in the context of ReLU units where we offset the input by a fixed value, either +1 or −1. Intriguingly, while we do observe

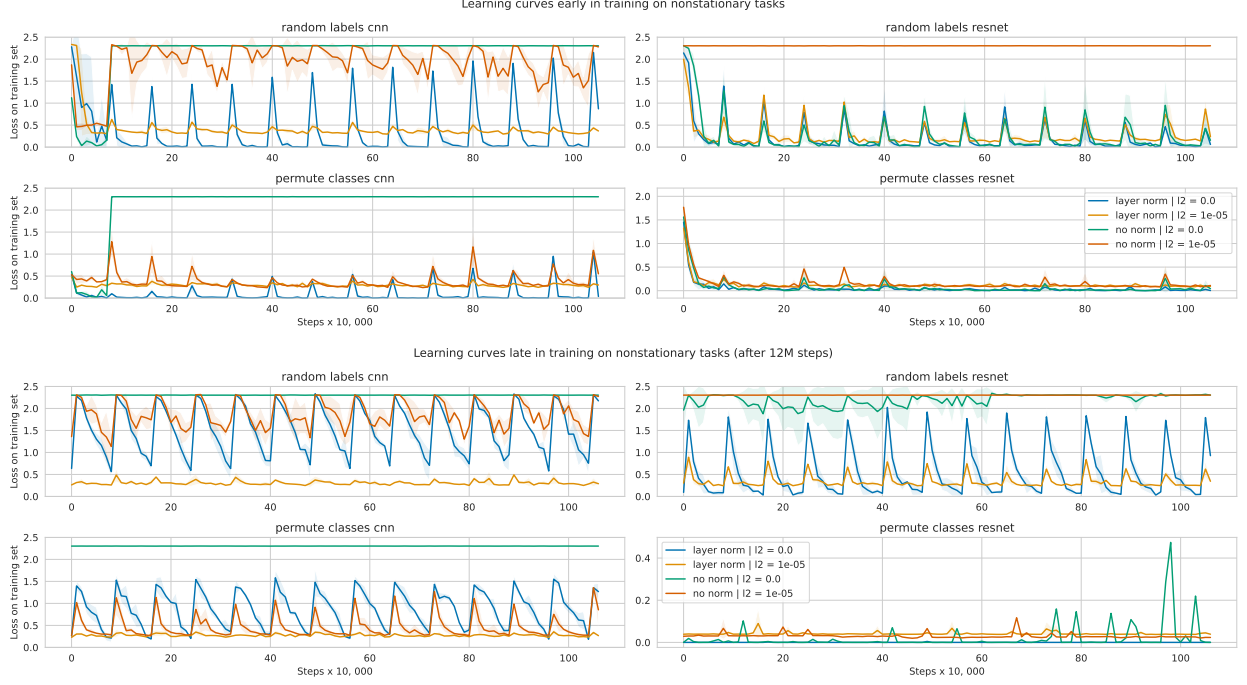


Figure 8: Learning curves early and late in a nonstationary training evaluation.

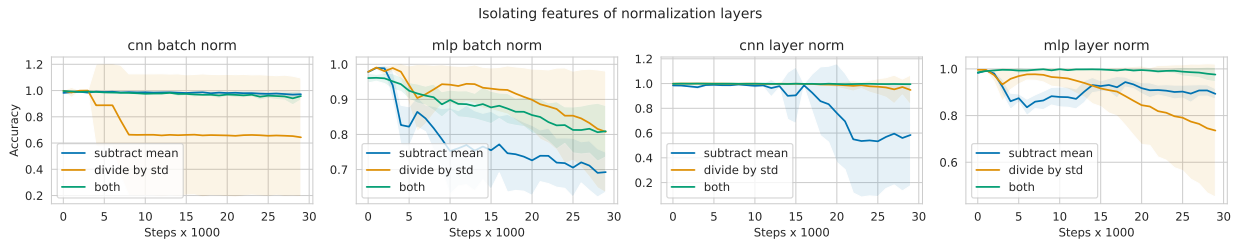


Figure 9: Ablating role of mean subtraction and normalization by standard deviation.

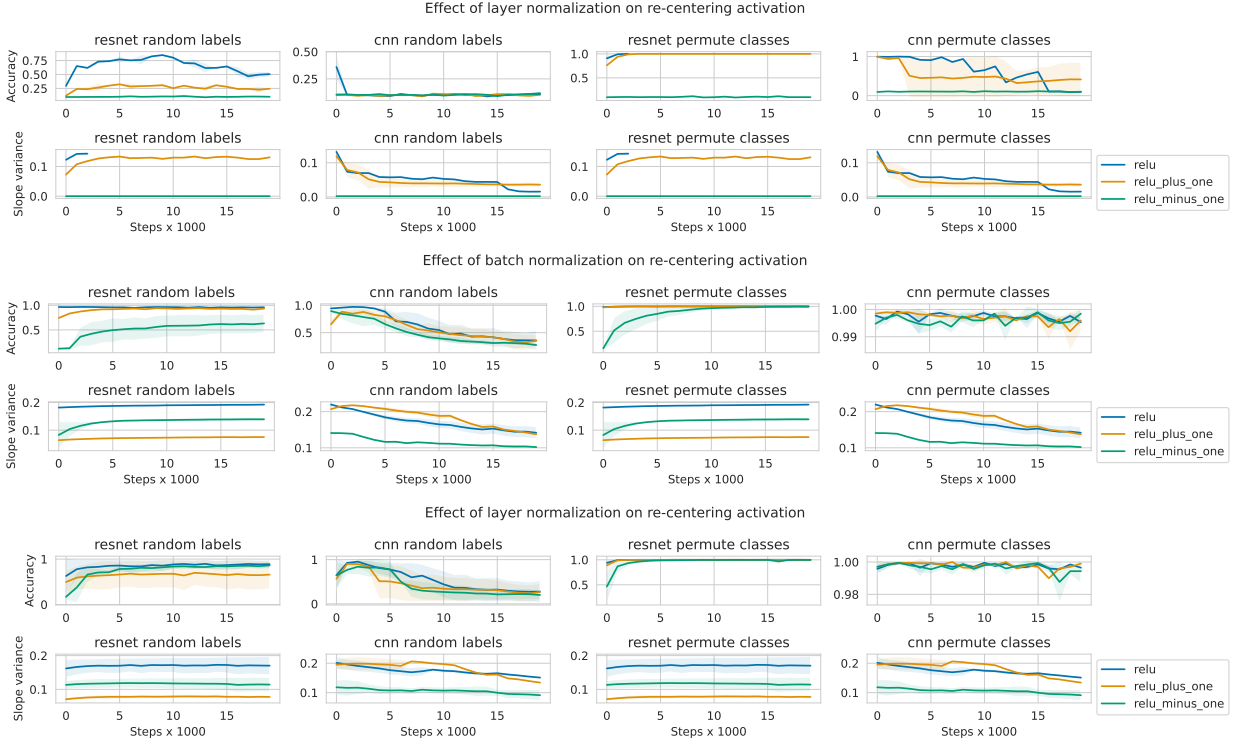


Figure 10: Effect of re-centering ReLU units on plasticity and expressivity.

a reduced variance in the slope of the nonlinearity when an off-center activation is used, this does not always result in reduced performance. Indeed, in many cases the normalized networks outperform an unnormalized network on the tasks despite reduced variability in the behaviour of the nonlinearity. This observation suggests that the role of normalization layers in maintaining plasticity may have less to do with the precise range in which they keep the activations, and more to do with the stability of that range.

E.4 EFFECT OF DEPTH AND WIDTH

In Figure 11 we investigate the relationship between network size and robustness to nonstationarity. We do not perform any additional hyperparameter sweeps for the different depths. Perhaps surprisingly, we find that in spite of using a learning rate that was tuned for a depth 4 network, using a depth of up to 12 results in improved performance in the permute-classes setting. In contrast, the deeper networks exhibit greater difficulty even in the first iteration of the random labels task. In contrast, and in agreement with the observations of Lyle et al. (2023), we do see consistent improvements from increasing width.

F ADDITIONAL RESULTS

F.1 OUTPUT SCALE AND PLASTICITY

We illustrate the effect of increasing γ on the plasticity of deep Q-learning agents in Figure 1. While increasing γ increases the scale of the TD targets, it also increases the degree of bootstrapping performed by the network, and it is difficult to disentangle these potentially competing mechanisms from the figure alone. To provide additional support for our claim that output scale can damage the plasticity of neural networks independent of bootstrapping, we distill the target phenomenon down to its most essential components. We perform standard mean squared error minimization on a set of random targets to which we add a fixed bias term. In our experiments, we use the CIFAR-10 dataset as inputs, and construct challenging regression targets by applying a high-frequency sinusoidal transformation to the output of a randomly initialized neural network. Specifically, for $M = 10^5$, we have $f_{\text{target}}(\mathbf{x}) = \sin(Mf_{\text{rand}}(\mathbf{x})) + b$, where b is some fixed bias. A one-dimensional interpretation of the regression targets for different values of b is provided in

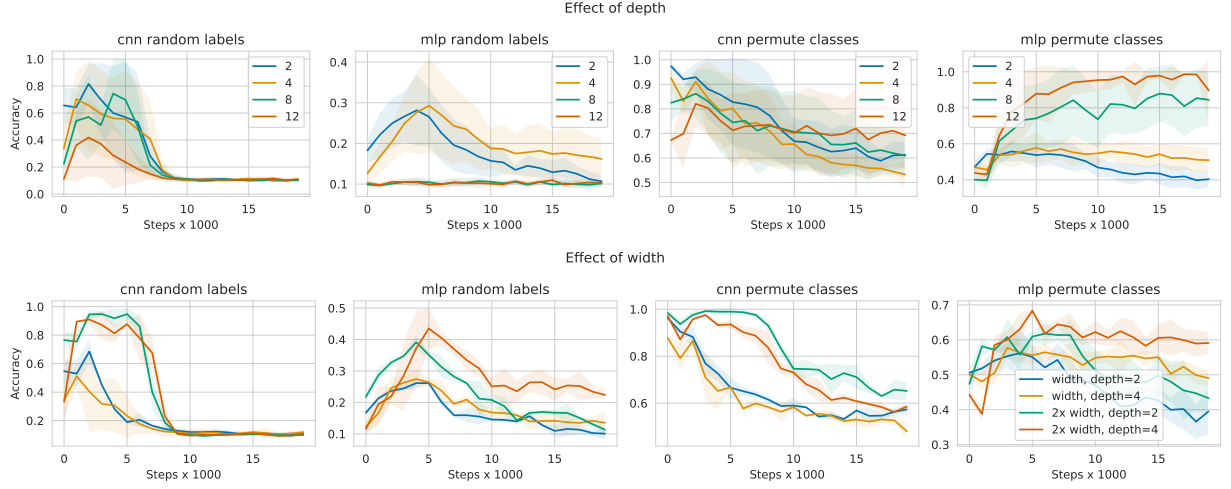


Figure 11: **Top:** relationship between depth and plasticity. We see that networks benefit from depth in the permute-classes task but not in the random-labels task, where increased depth without additional hyperparameter tuning reduces performance and accelerates the loss of plasticity. **Bottom:** relationship between width and plasticity. Across depths, increasing width is beneficial.

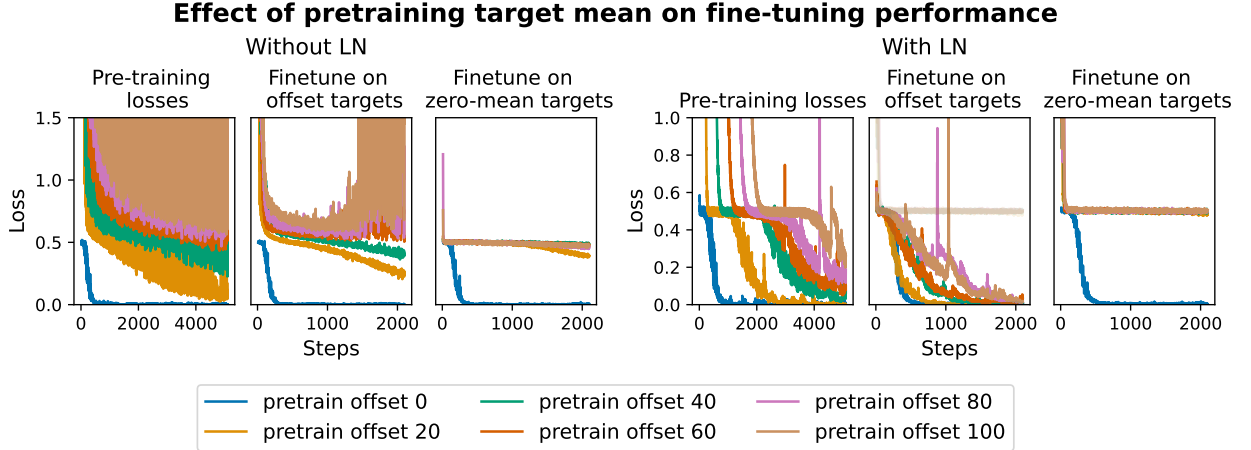


Figure 12: Detailed learning curves for networks with and without layer normalization when trained on regression tasks with varying pre-training offset means.

Figure 13. We provide more detailed learning curves for networks both with and without layer norm in the architecture from Figure 1 in Figure 12.

In our experimental framework, we pretrain a different convolutional network (the same CNN architecture as is used in the “standard” RL training regime) on a target function defined as above for each $b \in \{0, 8, 16, 32\}$. We then construct a new set of targets, which use a different random target network initialization and have either bias equal to the pretraining bias or $b = 0$. In both cases, we see that the network pretrained on $b = 0$ is better able to reduce the loss on the finetuning targets. We further see a monotone relationship between the size of the pretraining offset and the final loss of the network. Averaged learning curves over three random target seeds are shown in Figure 14, and averaged final losses (we average both over seeds and over the final 10 SGD iterates to reduce noise) are shown in Figure 13. We run the experiment on a subset of 10,000 images from the underlying dataset and use a batch size of 512; we use the Adam optimizer with learning rate 0.001.

Looking into the singular value decomposition of the penultimate-layer features of the network, obtained by performing SVD on a $n \times d$ matrix given by feature embeddings for n randomly sampled inputs with $n = 128$ in this case,

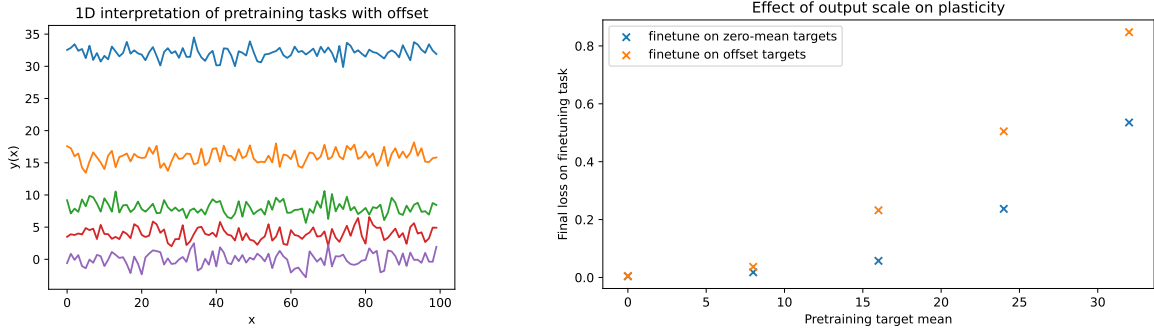


Figure 13: Left: Visualization of a 1D analogue the regression targets used for pre-training (actual targets are constructed over an image dataset to allow for more realistic network architectures). Right: “dose-response” curve for pretraining target offset scale vs final performance on fine-tuning task with either zero offset or the same offset as the pretraining data. In both cases, we see similar reductions in fine-tuning performance as a function of the pretraining offset.

Random regression targets without layer normalization

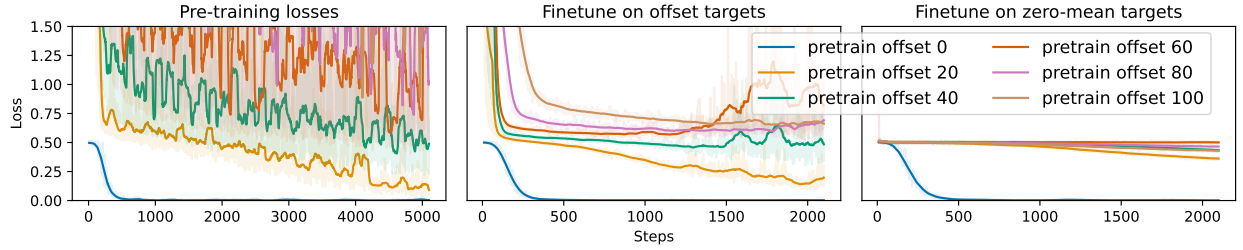


Figure 14: Effect of pretraining bias magnitude on finetuning accuracy. We omit the first 1000 steps of the fine-tuning period to allow for a more informative scale.

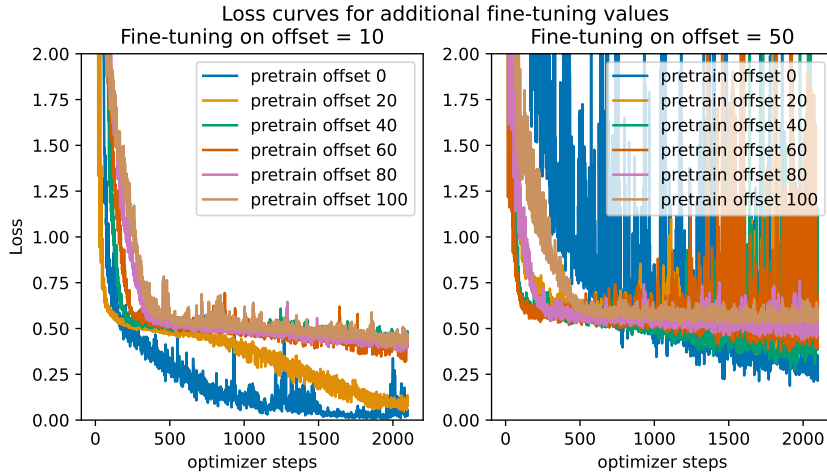


Figure 15: Additional targets on which to study the pretrain-finetune regression setting. We replicate the findings with zero-mean and equal-bias targets, observing greater instability with both increasing pretraining target magnitude and increasing fine-tuning target magnitude.

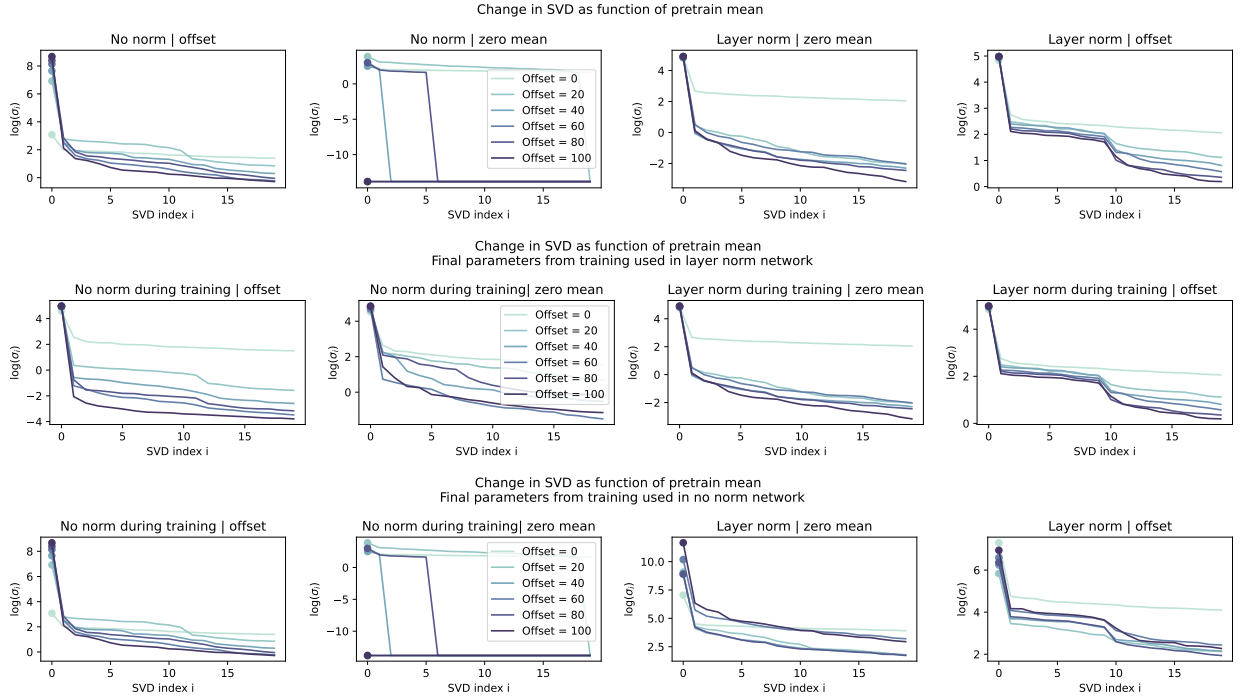


Figure 16: Distribution of top singular values as a function of index (note log scale). Magnitude of the top singular value grows with pretraining offset norm and remains high even after fine-tuning on We observe that networks with layer normalization have less extreme ill-conditioning of their features than networks without layer normalization.

gives some insight into what is driving the optimization difficulties in networks trained to minimize a regression loss on a target of the form $f(x) = 100 + \epsilon(x)$. In networks which don't include layer normalization, we see an immense increase in the maximal singular value as the pretraining target offset grows (ranging from $O(10^3)$ for mean-zero targets to $O(10^8)$ for mean-100 targets). Layer normalization constrains the maximum norm of the features in networks which incorporate it and thus the maximum singular value of the feature matrix, however even in these networks we see a significant decline in the magnitudes of lower-order singular values relative to the largest one.

And what exactly does this largest singular value correspond to in feature space? As we can see in Figure 17, this is the dimension in which the network is encoding the bias term of the targets. First, we note that a neural network with nonzero bias weights in its final output layer should in theory be able to represent functions with arbitrarily large means by increasing the corresponding bias term; however, this does not happen in practice. We observe in the rightmost plot of Figure 17 that the norm of the bias weights in these networks does not exhibit a monotone trend as the pretraining target offset grows. This is particularly striking in contrast to the trends observed for the feature embeddings, which grew not only monotonically but by several orders of magnitude.

Though their bias terms are small, these networks are nonetheless outputting functions whose mean corresponds to that of the pretraining targets, which means that a large constant function must emerge from somewhere. As it turns out, rather than encoding this mean in the bias weights, the networks have instead learned to produce features which can be used as effective bias terms, by encoding the bias term into a single-dimensional subspace of the feature space with which feature embeddings have roughly constant dot product. To study the contribution of this single-dimensional subspace of the embedding space to the network's output, we project the feature matrix onto that subspace and then apply the final layer weights to the projected matrix, visualizing the resulting projected outputs for 128 randomly sampled inputs as a (highly disordered) line plot. Strikingly, these visualizations make clear that the subspace encoded by the principal singular value is being used by these networks as essentially a bias term, to which perturbations arising from other subspaces are added. As target magnitude increases, so does the deviation from a perfectly constant function along with the relative contribution of lower singular vectors. Notably, in the networks which are fine-tuned on targets with mean zero, the corresponding dimension of feature space contributes almost zero to the final fine-tuned output, suggesting that the network is using this dimension as an offset term and overwrites it when this offset is no longer needed (though in the case of larger magnitudes this is accidentally achieved by saturating units, as we can see in the second row of Figure 17 that the network output is characterized entirely by the bias).

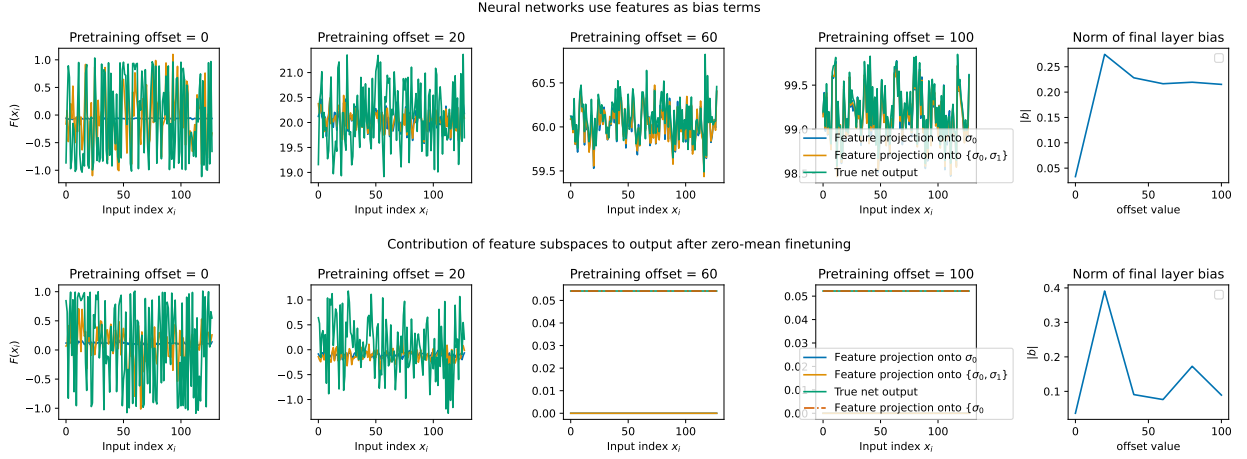


Figure 17: Visualization of contribution of principal components of penultimate layer features to the final network output, in contrast with the contribution of the final-layer bias term.

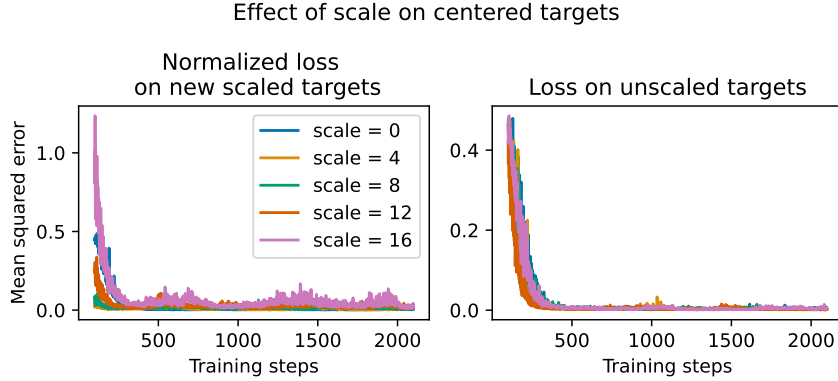


Figure 18: We contrast our previous analysis of offset targets with targets which are centered and scaled. Similar to Figure 13, we construct a pre-training task of the form $c + \alpha f_\theta(\mathbf{x}) + \sigma(\mathbf{x})$, where f_θ is the output of a randomly initialized network and σ is a high-frequency function. Rather than scaling c , we instead scale the smooth part of the function by a factor α . In contrast to the case of scaled c , we don't see an effect of pre-training scale on finetuning loss on $\alpha = 0$. While we do see an increased loss when finetuning on targets with larger α , this increase is in line with the natural increase in mean squared error obtained by scaling targets and predictions.

As a result, we see that enormous singular values and ill-conditioning of the feature embeddings is a direct consequence of outputting a large, near-constant value. This occurs in its most extreme form in networks trained without layer normalization: in networks with layer normalization, this ill-conditioning effect is somewhat mitigated but nonetheless still occurs. Although in theory a network could increase the magnitude of the bias term in its output layer to achieve a larger output mean, in practice networks encode a 'bias' term in their features instead.

In order to disentangle effect of output magnitude from output mean, we run an additional experiment involving mean zero targets that are scaled by different factors. The results from this experiment are given in Figure 18. From these we see that it's specifically a large target mean, and not necessarily large targets in general, that lead to plasticity loss.

F.2 UNEVEN PARAMETER NORM GROWTH

We illustrate that even in a small MLP, differential scaling of the input, hidden, and output layers can result in markedly different trends in the growth of the parameters in these layers. The network is trained on an extremely simple task: we fix some randomly generated inputs and targets $\mathbf{x}, \mathbf{y}$, and then regress the network on $\mathbf{y} + \sin(t/20)$ where t is

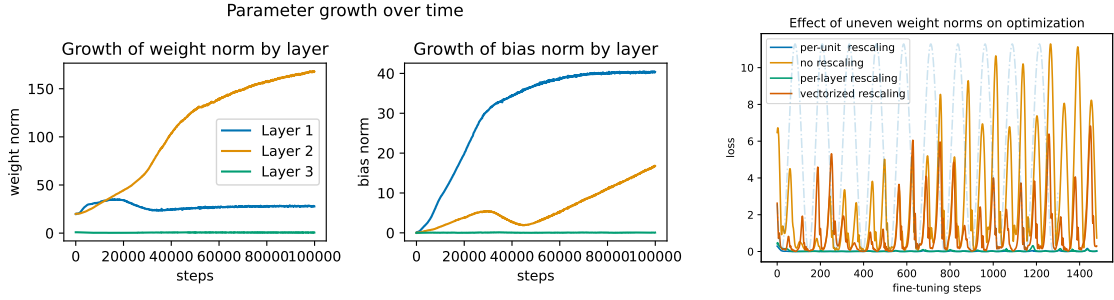


Figure 19: Parameter norm growth of different layers in an MLP trained on nonstationary regression

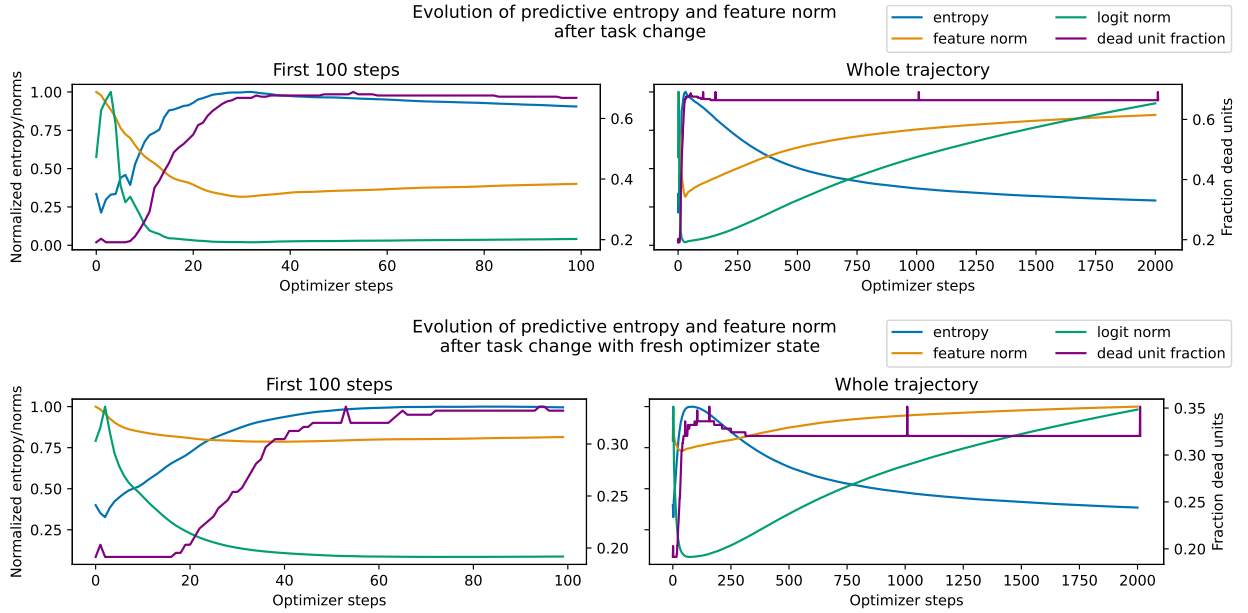


Figure 20: Evolution of logit norm, feature norm, logit norm, and dead unit fraction in a one-hidden-layer MLP trained to memorize one set of random MNIST labels, and then fine-tuned on a second set. Results show early and total training dynamics on the second set of random labels. Top: initial optimizer state and parameters taken from the final step of the previous task. Bottom: resets the adam optimizer before continuing training on the second set of random labels. Note the smaller right-hand-side y-axis scale in the bottom figure, corresponding to half the number of dead units.

the number of optimization steps taken so far. We also observe that this unequal growth can be accompanied by optimization difficulties if these networks are then trained on a new task.

F.3 DEATH OF A RELU NEURON

Figure 2 illustrates at a high level how early gradients are biased towards either reducing or increasing pre-activation magnitudes over all inputs. It is natural to then conclude that, because updates immediately after a task change can be extremely large due to out-of-date second-order estimates in adaptive optimizers such as Adam, this bias will have the effect of potentially killing off many units. In this section, we provide a step-by-step illustration of how units die off, and why sudden task changes are particularly hazardous for ReLU activations. We rely on the visualizations in Figure 20. Concretely, the following four factors drive the death of ReLU units:

1. Immediately after a task change, the network aims to increase the predictive entropy of its outputs. This can be seen in the increase in output entropy immediately following the task change, along with the corresponding decrease in logit norm over the first hundred optimizer steps.

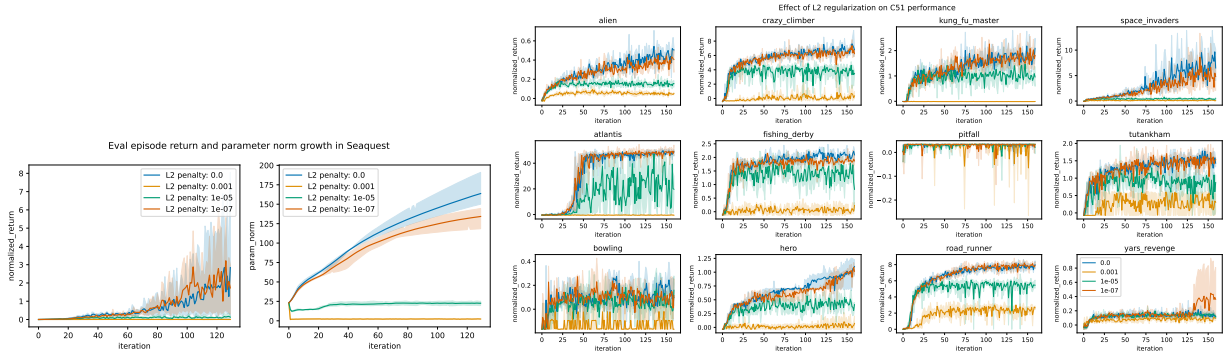


Figure 21: Parameter growth in Seaquest and effect of L2 regularization on the C51 agent in a subset of Atari.

2. There are two ways to reduce the magnitude of the incorrect class probability: one can either reduce the norm of the final-layer weight associated with that logit, or one can reduce the norm of the incoming features. As we can see, the network tends to decrease the norm of the incoming features.
3. Gradients which reduce the norm of the features have the effect of reducing the pre-activation value for most inputs, since negative pre-activations do not contribute to the feature norm due to the ReLU. We see that in many units the gradient has a negative dot product with all inputs.
4. If the optimizer state is not reset, then outdated second-order estimates cause large update steps. Large update steps in a direction which push down pre-activations quickly results in unit death, with all pre-activations becoming negative within a handful of steps.

F.4 DETAILED ATARI RESULTS

We provide per-game performance curves for the deep RL agents trained on the Atari benchmark in Figure 28, and also illustrate the nuances of applying weight regularization in this domain in Figure 21. As can be seen from these figures, layer normalization provides obvious benefits in the C51 agent, however the agent does not benefit from L2 regularization. Note that unlike the figure in the main body of the paper, our sweep over L2 regularization coefficients is only evaluated on a subset of the atari benchmark obtained by sorting the games in alphabetical order and selecting every fifth one. Our analysis of the seaquest environment shows that the parameter norm of these agents grows at a modest rate and is several orders of magnitude below the level which caused optimization challenges in the image classification tasks. We see that a penalty of $1e-7$ slightly slows the rate of parameter growth without interfering with performance, a value that is much lower than what we studied in the sequential supervised learning setting. This is consistent with prior observations that deep RL agents tend not to benefit from L2 regularization.

F.5 L2 REGULARIZATION SWEEP

Noting a small gap in peak performance between the models trained with layer normalization but either with or without L2 regularization, we investigate whether this gap might be caused by an overly-aggressive L2 penalty. Given the highly noisy nature of the learning targets in the case of random label memorization, we would expect that maintaining small weight norm would be in tension with reducing loss on the training set, and so may require a smaller penalty than the value of 10^{-5} that we used as an initial ‘reasonable guess’. We run this experiment on four architectures (we omit the ResNet due to the longer training time required and the short rebuttal period): a depth-4 CNN with fully-connected layer width 256 and either GeLU or ReLU activations, along with a depth-4 MLP with width 512, again using either GeLU or ReLU. We run each network configuration for 200 iterations of 200K steps, and use five different random seeds. Shaded region indicates standard deviation. As we see in Figure 22, reducing the L2 penalty by an order of magnitude and allowing for a slightly longer reset interval (200,000 steps) significantly reduces the gap between regularized and unregularized convolutional networks, while for MLPs we see that the $1e-5$ penalty is highly effective at maintaining plasticity while maintaining close to 100% final performance even after 200 resets (equating to 40M optimizer steps).

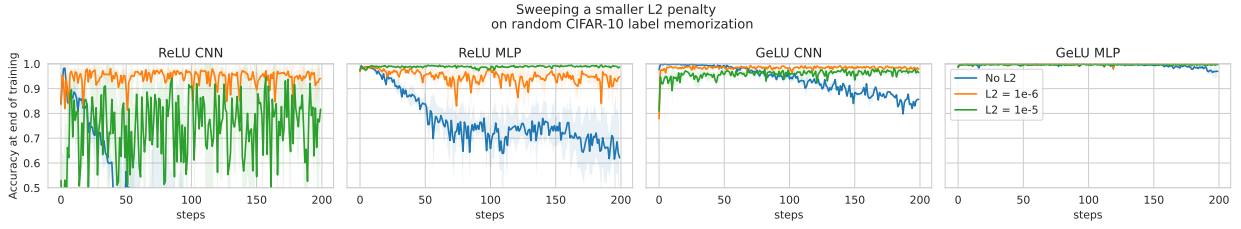


Figure 22: Comparison with a smaller L2 penalty: for suitable regularizer weights, we see minimal reduction in performance at the end of an iteration; using a larger-than-optimal regularizer can slow down training in some instances.

F.6 LONGER RUN OF CONTEXTUAL BANDIT TASK

Upon request from reviewers, we have included a longer run of the DQN agents trained on the image classification MDP in Figure 23. The agent is trained for 500,000 steps (10x the number of iterations shown in the main body of the paper) in total using the Deep Q-learning algorithm. We sweep over configurations of the following properties: L2 regularization (1e-6 or 0), regression or classification loss, layer normalization or no layer normalization, and either a ‘fast’ target network update period (500 optimizer steps) or a ‘slow’ period (5000 optimizer steps). Overall, we see that the networks trained without layer normalization, weight decay, or classification losses quickly saturate at the level of random guessing on the probe task. However, combining these approaches results in significantly better long-term performance on the probe tasks, in some cases resulting in positive forward transfer. There is some variation across configurations whereby networks may see improved or reduced performance on the probe task compared to its value at initialization, but this value tends to stabilize quickly in networks trained in the most stable regimes. In all experiments which use the classification loss, we include a label smoothing parameter of 0.1.

F.7 WILDS DATASET BASELINE

We also include an additional evaluation of the iwildcam experiment including as a baseline a network which is re-initialized at each task change. Including this baseline, we see that in fact the networks trained with layer normalization not only do not exhibit plasticity loss on this task distribution, but also exhibit positive forward transfer and are able to outperform the reinitialized baseline.

F.8 UNIT LINEARIZATION IN RESIDUAL NETWORKS

While we provide a hint that linearized or “zombie” units may be more informative than dormant or “dead” units in neural networks in Figure 2, in fact the most compelling evidence that this phenomenon may be tied to plasticity can be observed in residual networks. The ResNet18 architecture we use is significantly deeper than the CNN and MLP, and although it features skip connections which in theory should serve as a buffer against poor signal propagation through blocks, we observe that it tends to be more sensitive to architectural factors such as layer norm than the shallower architectures. This suggests that it is also more sensitive to signal propagation failures, a feature that becomes apparent when we consider a wider range of activation functions. In Figure 25, we consider the ResNet18 architecture under a variety of experimental conditions on the sequential image classification regime. In particular, we consider four activation functions, two configurations of LayerNorm, and two types of nonstationarity.

We see an intriguing correlation between trends in the number of linearized units in the network and the ability of the network to reduce its loss on later tasks: networks with a large number of effectively linear units in some cases are never able to outperform random guessing, even if the network uses a non-saturating activation function and so does not suffer from dormant neurons. Further, within a trajectory, increases in the number of zombie units line up with periods where the network loss also increases, while periods where quantity declines correspond with phases where the network improves its loss (a particularly striking example of this is the case of the absolute value function with random labels and layer normalization). While raw values of the number of linearized units are not necessarily useful for cross-architecture comparisons, it is highly informative within a trajectory, and exhibits significantly stronger correlations with performance even than the number of dead units in the penultimate feature layer.

We also conduct a toy experiment in a small MLP trained on a subset of MNIST with canonical labels in Figure 26. We first randomly initialize the network and train on the subset. We then take the same initialization and map every parameter to its absolute value. As a result, every unit in the second layer of the network is a ‘zombie’ unit: it is

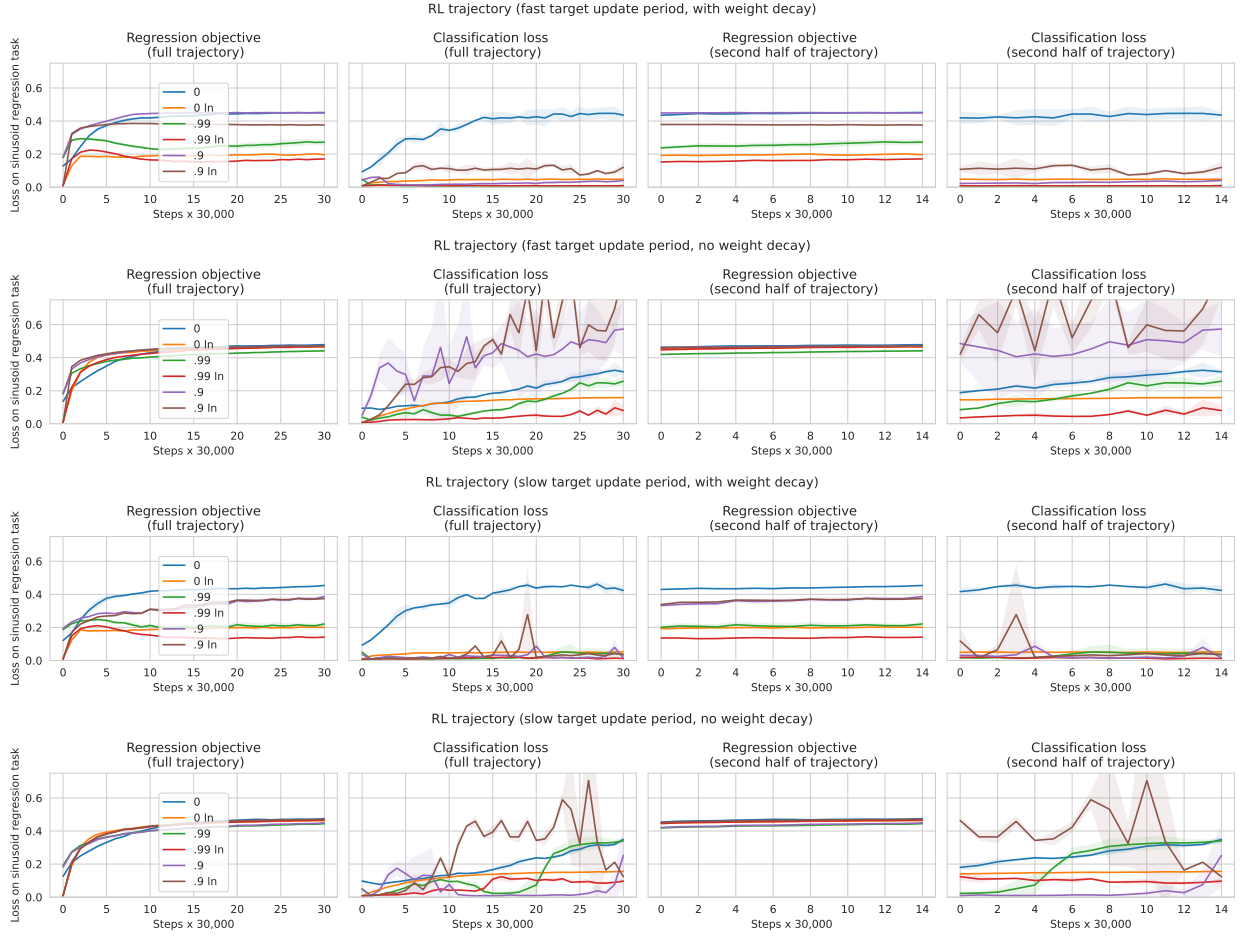


Figure 23: Evolution of plasticity under longer runs on the image classification MDP.

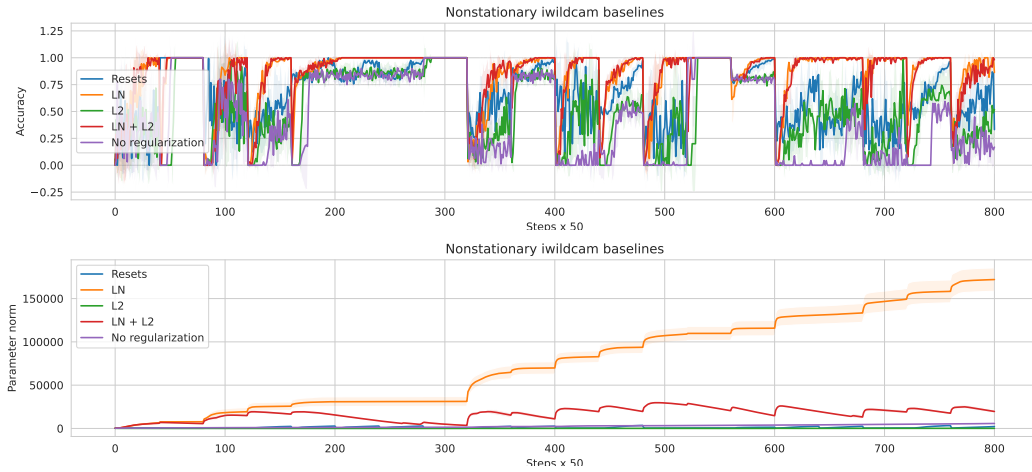


Figure 24: We see that using layer normalization allows networks to maintain plasticity and exhibit positive forward transfer on the natural distribution shift problem. The network trained with layer normalization but without L2 regularization exhibits greater parameter norm growth than the other networks, but the training run is sufficiently short that this does not manifest in training difficulties.

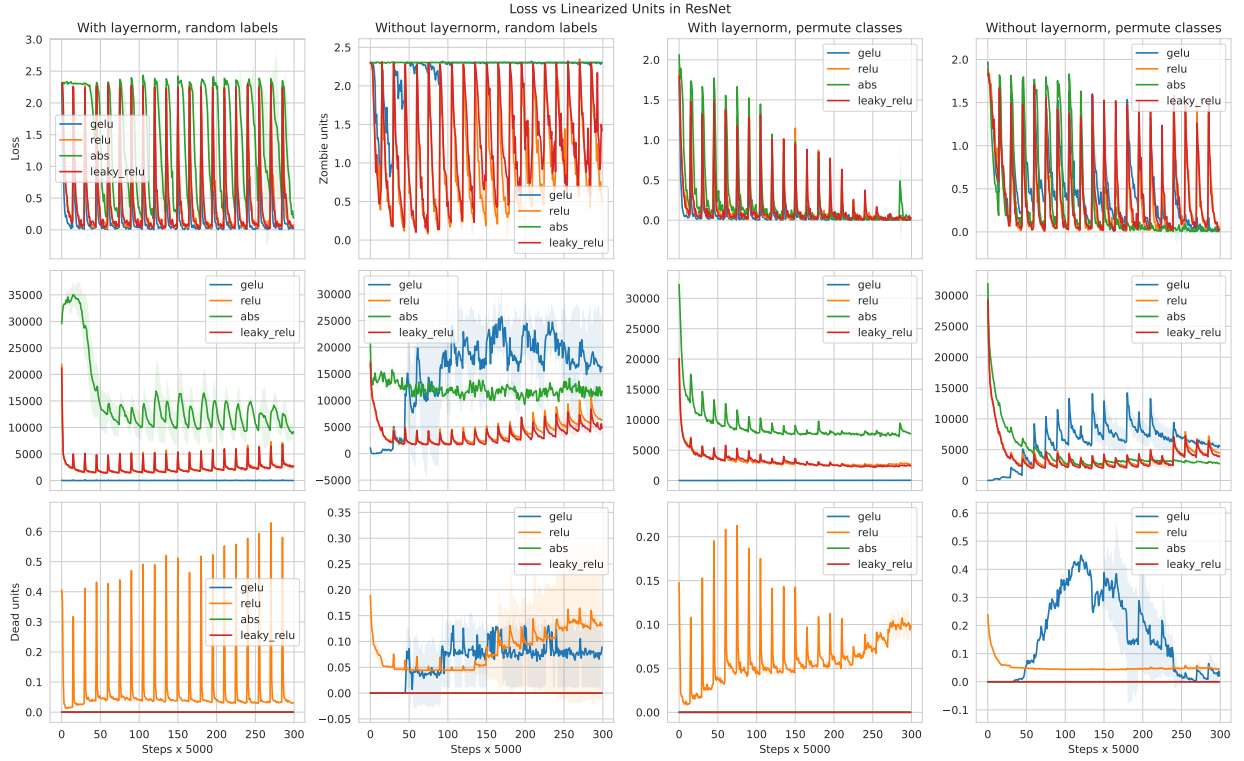


Figure 25: Loss of plasticity in our ResNet architecture trained on sequential image classification tasks. We see that high numbers of zombie tend to correlate with networks that are unable to reduce their loss, even in networks with non-saturating activation functions such as leaky ReLU and absolute value activations. In contrast, dormant or dead neurons are only observable in the ReLU and GeLU networks, providing an incomplete picture of signal propagation failures in the network.

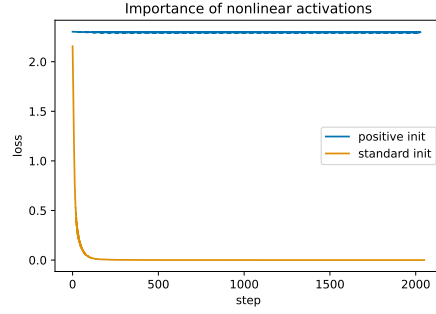


Figure 26: Effect of initializing a network to contain only zombie units in its second layer



Figure 27: Ablations on the axes on which we apply different normalization transformations. We see that applying only the mean subtraction component of batch normalization along with only the standard deviation division of layer normalization results in greater robustness across architectures than applying standard layer or batch normalization.

still able to propagate gradients, but only applies a linear transformation to its inputs. We find that starting from this initialization, the performance of the network never decreases even after two thousand optimizer steps.

F.9 NORMALIZATION METHOD ABLATIONS

Based on the framework outlined in Figure 1, we would expect layer and batch normalization to operate on subtly different mechanisms, and therefore potentially to provide even greater benefits when used in combination. In particular, we would expect that batch normalization would provide particular benefits due to avoiding saturated units and centering the distribution of preactivations to every nonlinearity about zero, which for most activation functions is exactly where the function is most nonlinear, thus avoiding the zombie unit phenomenon. This mechanism is independent of the ‘normalization’ component of batch norm (i.e. the division by the standard deviation of the preactivations). In contrast, while layer normalization will avoid pathologies where *all* units saturate, it will have a much weaker effect on this mechanism. We would expect that the benefits of layer normalization would primarily stem from its hard constraint on the feature norm, which will have beneficial effects on signal propagation (Martens et al., 2021) along with a normalizing effect on the feature gradients (Xu et al., 2019). One might then expect that a more effective strategy for maintaining plasticity, at least in supervised image classification tasks which are known to be robust to the stochasticity induced by batch normalization, would be to apply different normalization transformations along different axes of the features.

We explore some variants of this idea in Figure 27, where we follow the nonstationary image classification protocol described previously, using the random label memorization variant. We train each network for 50,000 steps between re-randomizations and use an L2 penalty of $1e-5$. We train for 100 iterations. We see architecture-dependent effects, where for example the ResNet is not able to train in the absence of the feature norm constraint from layer normalization while the MLP struggles in the absence of mean subtraction from batch normalization.

In the case of the MLP, we observed in later experiments that centering the inputs at approximately zero was important for training the network, and that most of the benefits of the mean subtraction from batch normalization seen here were replicated by input centering. However, we do consistently observe that across architecture, the combination of division by standard deviation along the feature axis and mean subtraction along the batch axis slightly outperforms naive layer and batch normalization. Ultimately, because the gains we saw from this nonstandard combination were relatively minor in most architectures, and because batch normalization is known to provide much less benefit in RL or natural language tasks than it does in image classification, we opted to use standard layer normalization in most of our experiments.

F.10 PER-GAME C51 AND RAINBOW AGENT RESULTS

We include a per-game visualization of the C51 agent’s performance with and without layer normalization. We include a second variant not included in the main paper which used both batch and layer normalization, noting that our findings corroborate prior observations that batch normalization can hinder performance in deep RL as seen in Figure 28. We also look at the Rainbow agent with and without layer normalization in Figure 29.

F.11 DEEPMIND CONTROL SUITE EXPERIMENT DETAILS

To complement our findings on Atari, we also consider a SAC agent trained on continuous control tasks from the DeepMind Control suite. This benchmark significantly differs from Atari along a number of axes: the action space is continuous, the input space is low-dimensional, and the network architectures typically used in this domain are fully-connected rather than convolutional. Somewhat surprisingly, we nonetheless see significant benefits from the incorporation of layer normalization into the architecture we used. We did not perform extensive hyperparameter tuning on this domain.

Algorithm: we use the Soft Actor Critic algorithm (Haarnoja et al., 2018).

Network: for the critic architecture, we use a three hidden layer MLP encoder with hidden dimension 256, and put LayerNorm prior to the activations of the first- and third-layer outputs; the output of this encoder is then fed into a two-layer MLP with elu activations. The actor network architecture uses the same encoder architecture with a single 256-dimensional hidden layer prior to the Gaussian policy output head.

Training protocol: we train each agent for 1B environment frames and use five seeds for each experiment configuration. We use the adam optimizer with a learning rate of $3e-4$ for the actor and $1e-4$ for the critic.

Domains: we evaluate on 28 domains from the DeepMind control suite.

Results: we observe significant gains from layer normalization in several environments where the network constructed without normalization layers fails to make any learning progress.

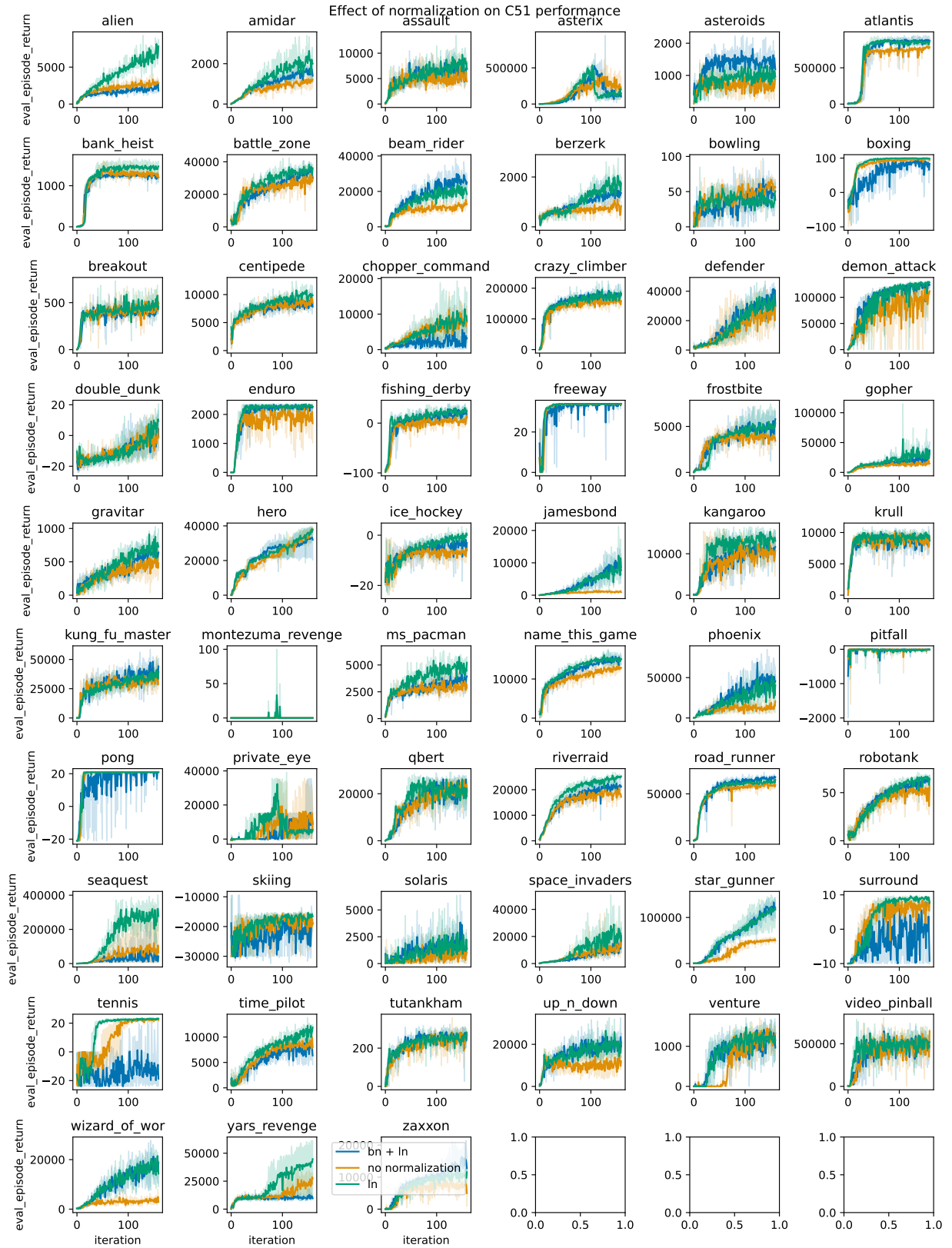


Figure 28: Per-game results for C51 agent with layer normalization or a combination of batch and layer normalization.

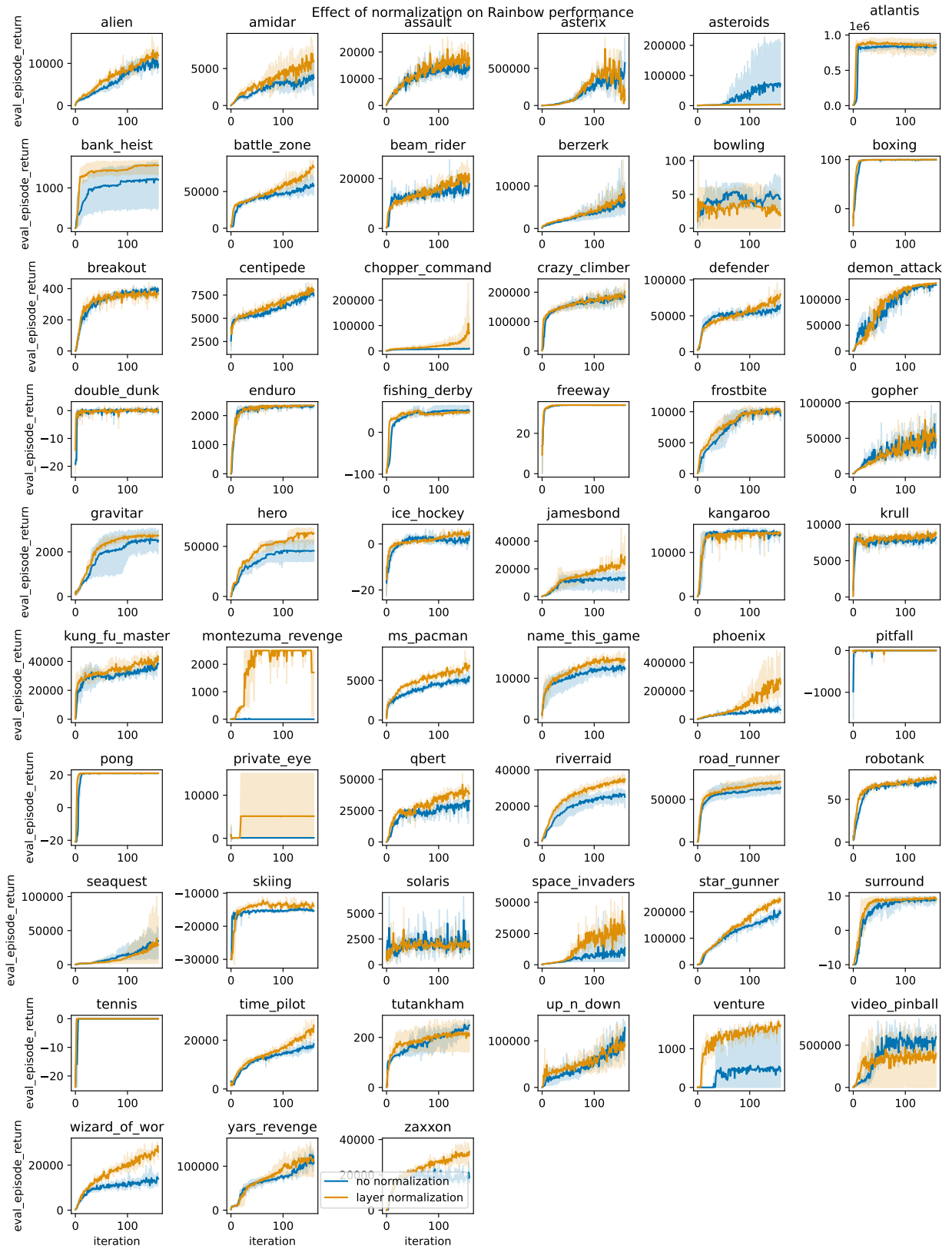


Figure 29: Per-game results for C51 agent with layer normalization or a combination of batch and layer normalization.

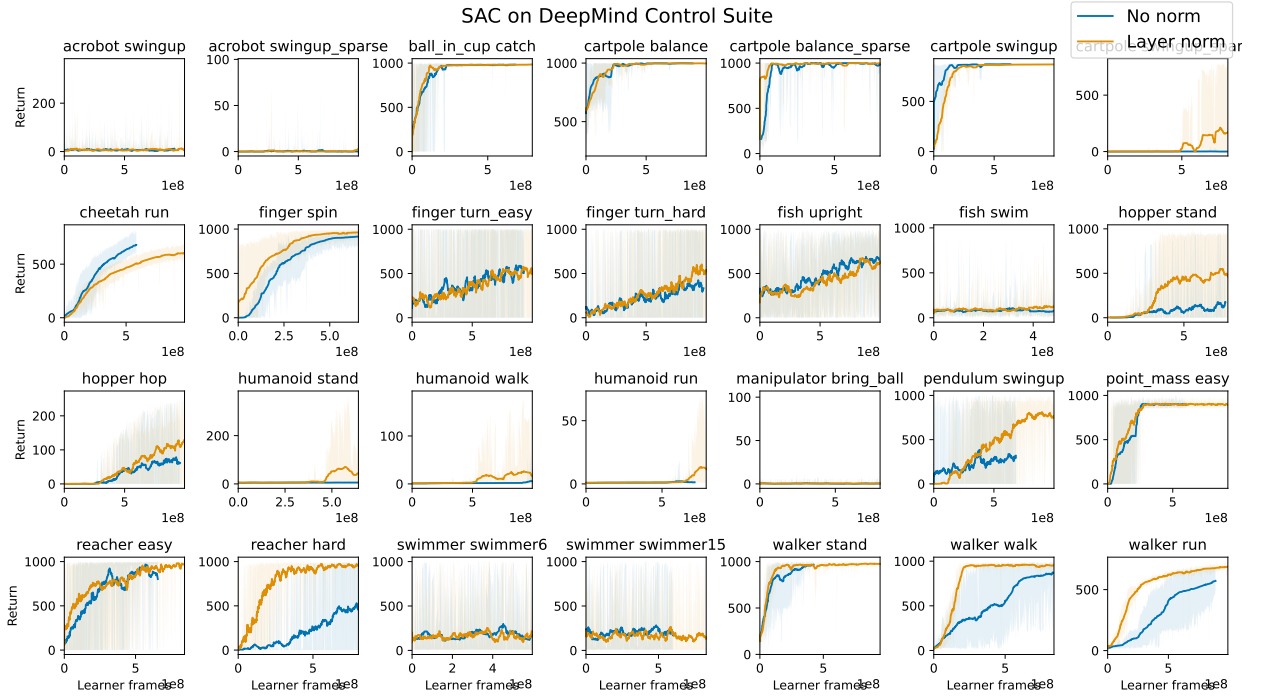


Figure 30: Effect of layer normalization in DeepMind Control Suite.

G A DISCUSSION ON THE INDEPENDENCE OF CAUSAL MECHANISMS

This paper claims that the mechanisms we discuss in Section 3.2 are *independent*. To justify this claim, it is first necessary to clarify what we mean by independence. In this paper we will say that *two mechanisms of plasticity loss are independent if there exist learning problems where completely mitigating one mechanism does not prevent plasticity loss due to the other*. A stronger notion of independence might require that the mechanisms not interact, however in a system as complex as training a deep neural network this is an excessively stringent requirement.

Given this definition of independence, it is then straightforward to determine whether two mechanisms are independent, at least if the answer is positive: it suffices to identify a learning problem constructed so that one mechanism cannot occur, and where the other can be shown to cause the network to lose plasticity. We can in fact find such learning problems within the experiments presented in previous sections, which we now enumerate.

While **preactivation shift** and **weight norm** often correlate in neural network training trajectories, much of this correlation is a natural result of neural network optimization, as both weights and features must necessarily change over the course of learning, and any two quantities which consistently trend away from their initial values may look correlated. To see that the mechanisms by which these phenomena influence plasticity are independent, we first consider a network where the weight norm is regularized via weight decay and can be verified not to grow uncontrollably, but where preactivation shift causes performance degradation. We find such an example in Figure 24, where the network trained with L2 regularization exhibits declining performance, while the network trained with L2 + layer normalization does not. Similarly, the CNN shown in Figure 8 exhibits loss of plasticity in long training trajectories when only layer normalization is applied, but not when layer normalization is used in conjunction with L2 regularization.

In contrast, the relationship between preactivation shift and **unit saturation** is more of a direct causal arrow: it is possible for pathologies to arise as a result of preactivation shift that do not correspond to unit death, for example the linearized units illustrated in Figure 19; however, unit death requires a shift in the distribution of preactivations in order to occur (a counter example would be when units are dormant at initialization, however in this case the problem is not loss of plasticity but rather its nonexistence). Intriguingly, the results of Figure 21 do suggest that, all else being equal, a shift in the distribution of preactivations which preserves the per-unit mean over samples is less damaging than one which preserves the per-sample mean over units, consistent with the unsurprising conclusion that zeroing out gradient flow through a feature is more damaging than other more subtle signal propagation issues.

Regression target magnitude is a unidirectional factor in plasticity loss, as in most cases the magnitude of the regression targets is not caused by other optimization pathologies in the network, but rather is an unavoidable feature of the learning problem. The magnitude of regression targets can exacerbate **preactivation shift**, as large targets require commensurate changes in the network to fit. However, we see that even in networks which feature layer normalization, for which the preactivation mean and variance are fixed, we still see reduced ability to adapt to new tasks as a function of pretraining target magnitude in Figure 14. Similarly, while regression target magnitude can exacerbate **weight norm** growth, only adding L2 regularization is insufficient to mitigate the issue, as we can see in the RL agents in Figure 3 (RHS); similarly, the incomplete success of layer normalization at improving plasticity in this domain suggests that avoiding unit saturation across an entire layer is insufficient to mitigate plasticity loss caused by large target magnitudes.

Weight norm growth: it remains to show the independence of parameter norm growth and number of dead units as causes of plasticity loss. This is a relatively straightforward task: weight norm growth is often most pronounced in networks with layer normalization (see e.g. Figure 4), which avoid at least the most extreme version of unit dormancy where an entire layer becomes dormant. In the opposite direction, unit saturation is a highly effective means of reducing the growth of the parameter norm as it prevents gradients from acting on parameters in a way that could increase their magnitude, however has obvious deleterious effects on trainability.